

Explicit Allocation of Fixed-Sized Blocks

The simplest form of dynamic allocation involves blocks of a fixed size. By linking the blocks in a list, as in Fig. 7.41, allocation and deallocation can be done quickly with little or no storage overhead.

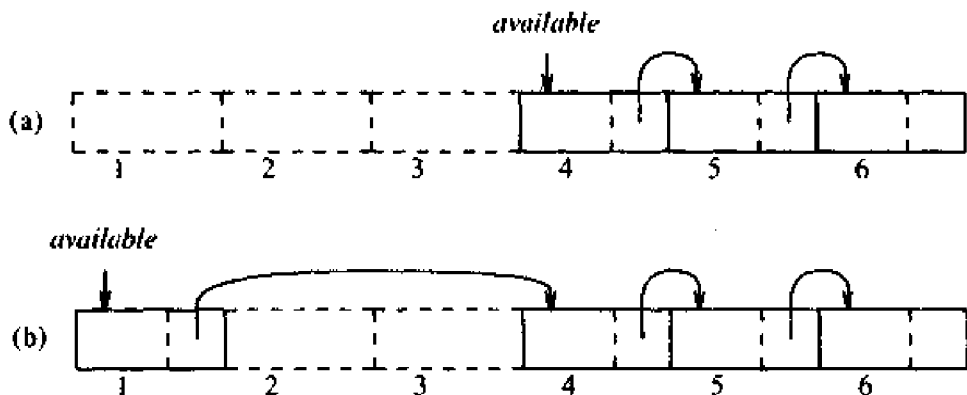


Fig. 7.41. A deallocated block is added to the list of available blocks.

Suppose that blocks are to be drawn from a contiguous area of storage. Initialization of the area is done by using a portion of each block for a link to the next block. A pointer *available* points to the first block. Allocation consists of taking a block off the list and deallocation consists of putting the block back on the list.

The compiler routines that manage blocks do not need to know the type of object that will be held in the block by the user program. We can treat each block as a variant record, with the compiler routines viewing the block as consisting of a link to the next block and the user program viewing the block as being of some other type. Thus, there is no space overhead because the user program can use the entire block for its own purposes. When the block is returned, then the compiler routines use some of the space from the block itself to link it into the list of available blocks, as shown in Fig. 7.41.

Explicit Allocation of Variable-Sized Blocks

When blocks are allocated and deallocated, storage can become *fragmented*; that is, the heap may consist of alternate blocks that are free and in use, as in Fig. 7.42.

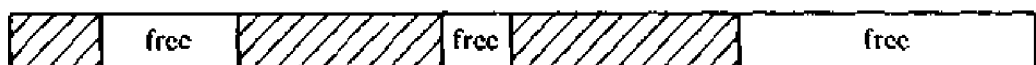


Fig. 7.42. Free and used blocks in a heap.

The situation shown in Fig. 7.42 can occur if a program allocates five blocks and then deallocates the second and fourth, for example. Fragmentation is of no consequence if blocks are of fixed size, but if they are of variable size, a situation like Fig. 7.42 is a problem, because we could not allocate a block larger than any one of the free blocks, even though the space is available in principle.

One method for allocating variable-sized blocks is called the *first-fit method*. When a block of size s is allocated, we search for the first free block that is of size $f \geq s$. This block is then subdivided into a used block of size s , and a free block of size $f - s$. Note that allocation incurs a time overhead because we must search for a free block that is large enough.

When a block is deallocated, we check to see if it is next to a free block. If possible, the deallocated block is combined with a free block next to it to create a larger free block. Combining adjacent free blocks into a larger free block prevents further fragmentation from occurring. There are a number of subtle details concerning how free blocks are allocated, deallocated, and maintained in an available list or lists. There are also several tradeoffs between time, space, and availability of large blocks. The reader is referred to Knuth [1973a] or Aho, Hopcroft, and Ullman [1983] for a discussion of these issues.

Implicit Deallocation

Implicit deallocation requires cooperation between the user program and the run-time package, because the latter needs to know when a storage block is no longer in use. This cooperation is implemented by fixing the format of storage blocks. For the present discussion, suppose that the format of a storage block is as in Fig. 7.43.

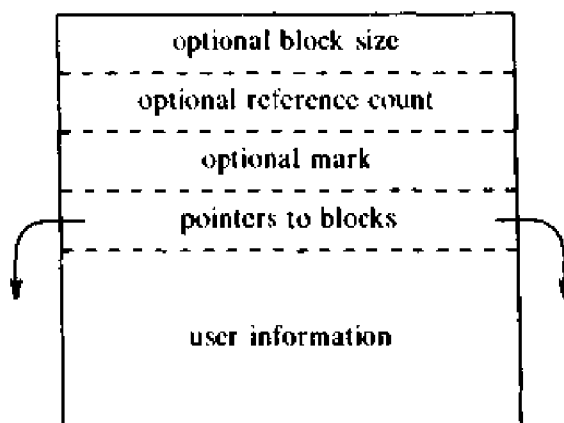


Fig. 7.43. The format of a block.

The first problem is that of recognizing block boundaries. If the size of blocks is fixed, then position information can be used. For example, if each block occupies 20 words, then a new block begins every 20 words. Otherwise,

in the inaccessible storage attached to a block we keep the size of a block, so we can determine where the next block begins.

The second problem is that of recognizing if a block is in use. We assume that a block is in use if it is possible for the user program to refer to the information in the block. The reference may occur through a pointer or after following a sequence of pointers, so the compiler needs to know the position in storage of all pointers. Using the format of Fig. 7.43, pointers are kept in a fixed position in the block. Perhaps more to the point, the assumption is made that the user-information area of a block does not contain any pointers.

Two approaches can be used for implicit deallocation. We sketch them here; for more details see Aho, Hopcroft, and Ullman [1983].

1. *Reference counts.* We keep track of the number of blocks that point directly to the present block. If this count ever drops to 0, then the block can be deallocated because it cannot be referred to. In other words, the block has become garbage that can be collected. Maintaining reference counts can be costly in time; the pointer assignment $p := q$ leads to changes in the reference counts of the blocks pointed to by both p and q . The count for the block pointed to by p goes down by one, while that for the block pointed to by q goes up by one. Reference counts are best used when pointers between blocks never appear in cycles. For example, in Fig. 7.44, neither block is accessible from any other, so they are both garbage, but each has a reference count of one.
2. *Marking techniques.* An alternative approach is to suspend temporarily execution of the user program and use the frozen pointers to determine which blocks are in use. This approach requires all the pointers into the heap to be known. Conceptually, we pour paint into the heap through these pointers. Any block that is reached by the paint is in use and the rest can be deallocated. In more detail, we go through the heap and mark all blocks *unused*. Then, we follow pointers marking as *used* any block that is reached in the process. A final sequential scan of the heap allows all blocks still marked *unused* to be collected.



Fig. 7.44. Garbage cells with nonzero reference counts.

With variable-sized blocks, we have the additional possibility of moving used storage blocks from their current positions.⁸ This process, called

⁸ We could do so with fixed-size blocks, but no advantage results.

compaction moves all used blocks to one end of the heap, so that all the free storage can be collected into one large free block. Compaction also requires information about the pointers in blocks because when a used block is moved, all pointers to it have to be adjusted to reflect the move. Its advantage is that afterwards fragmentation of available storage is eliminated.

7.9 STORAGE ALLOCATION IN FORTRAN

Fortran was designed to permit static storage allocation, as in Section 7.3. However, there are some issues, such as the treatment of **COMMON** and **EQUIVALENCE** declarations, that are fairly special to Fortran. A Fortran compiler can create a number of *data areas*, i.e., blocks of storage in which the values of objects can be stored. In Fortran, there is one data area for each procedure and one data area for each named **COMMON** block and for blank **COMMON**, if used. The symbol table must record for each name the data area in which it belongs and its offset in that data area, that is, its position relative to the beginning of the area. The compiler must eventually decide where the data areas go relative to the executable code and to one another, but this choice is arbitrary, since the data areas are independent.

The compiler must compute the size of each data area. For the data areas of the procedures, a single counter suffices, since their sizes are known after each procedure is processed. For **COMMON** blocks, a record for each block must be kept during the processing of all procedures, since each procedure using a block may have its own idea of how big the block is, and the actual size is the maximum of the sizes implied by the various procedures. If procedures are separately compiled, a link editor must be used to select the size of the **COMMON** block to be the maximum of all such blocks with the same name among the pieces of code being linked.

For each data area the compiler creates a *memory map*, which is a description of the contents of the area. This "memory map" might simply consist of an indication, in the symbol-table entry for each name in the area, of its offset in the area. We need not necessarily have an easy way of answering the question, "What are all the names in this data area?" However, in Fortran we know the answer for the procedures' data areas, since all names declared in a procedure that are not **COMMON** or equivalenced to a **COMMON** name are in the procedure's data area. **COMMON** names can have their symbol-table entries linked, with one chain for each **COMMON** block, in the order of their appearance in the block. In fact, as the offsets of names in the data area cannot always be determined until the entire procedure is processed (Fortran arrays can be declared before their dimensions are declared), it is necessary that these chains of **COMMON** names be created.

A Fortran program consists of a main program, subroutines, and functions (we call them all *procedures*). Each occurrence of a name has a scope consisting of one procedure only. We can generate object code for each procedure upon reaching the end of that procedure. If we do so, it is possible that most

of the information in the symbol table can be expunged. We need only preserve those names that are external to the routine just processed. These are names of other procedures and of common blocks. These names may not truly be external to the entire program being compiled, but must be preserved until the entire collection of procedures is processed.

Data in COMMON Areas

We create for each block a record giving the first and last names, belonging to the current procedure, that are declared to be in that COMMON block. When processing a declaration like

```
COMMON /BLOCK1/ NAME1, NAME2
```

the compiler must do the following.

1. In the table for COMMON block names, create a record for BLOCK1, if one does not already exist.
2. In the symbol-table entries for NAME1 and NAME2, set a pointer to the symbol-table entry for BLOCK1, indicating that these are in COMMON and members of BLOCK1.
3. a) If the record has just now been created for BLOCK1, set a pointer in that record to the symbol-table entry for NAME1, indicating the first name in this COMMON block. Then, link the symbol-table entry for NAME1 to that for NAME2, using a field of the symbol table reserved for linking members of the same COMMON block. Finally, set a pointer in the record for BLOCK1 to the symbol-table entry for NAME2, indicating the last found member of that block.
b) If, however, this is not the first declaration of BLOCK1, simply link NAME1 and NAME2 to the end of the list of names for BLOCK1. The pointer to the end of the list for BLOCK1, appearing in the record for BLOCK1, is updated of course.

After a procedure has been processed, we apply the equivalencing algorithm, to be discussed shortly. We may discover that some additional names belong in COMMON because they are equivalenced to names that are themselves in COMMON. We shall find that it is not actually necessary to link such a name XYZ to the chain for its COMMON block. A bit in the symbol-table entry for XYZ is set, indicating that XYZ has been equivalenced to something else. A data structure to be discussed will then give the position of XYZ relative to some name actually declared to be in COMMON.

After performing the equivalence operations, we can create a memory map for each COMMON block by scanning the list of names for that block. Initialize a counter to zero, and for each name on the list, make its offset equal to the current value of the counter. Then, add to the counter the number of memory units taken by the data object denoted by the name. The COMMON

block records can then be deleted and the space reused by the next procedure.

If a name **XYZ** in **COMMON** is equivalenced to a name not in **COMMON**, we must determine the maximum offset from the beginning of **XYZ** for any word of storage needed for any name equivalenced to **XYZ**. For example, if **XYZ** is a real, equivalenced to **A(5,5)**, where **A** is a 10×10 array of reals, **A(1,1)** appears 44 words before **XYZ** and **A(10,10)** appears 55 words after **XYZ**, as shown in Fig. 7.45. The existence of **A** does not affect the counter for the **COMMON** block; it is only incremented by one word when **XYZ** is considered, independent of what **XYZ** is equivalenced to. However, the end of the data area for the **COMMON** block must be far enough away from the beginning to accommodate the array **A**. We therefore record the largest offset, from the beginning of the **COMMON** block, of any word used by a name equivalenced to a member of that block. In Fig. 7.45, that quantity must be at least the offset of **XYZ** plus 55. We also check that the array **A** does not extend in front of the beginning of the data area; that is, the offset of **XYZ** must be at least 44. Otherwise, we have an error and must produce a diagnostic message.

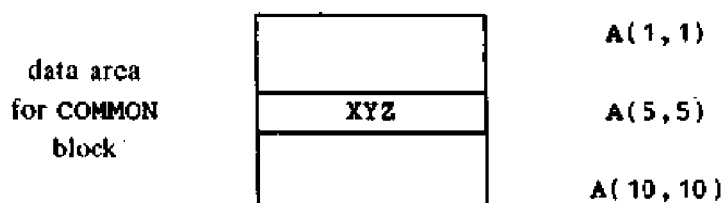


Fig. 7.45. Relation between **COMMON** and **EQUIVALENCE** statements.

A Simple Equivalence Algorithm

The first algorithms for processing equivalence statements appeared in assemblers rather than compilers. Since these algorithms can be a bit complex, especially when interactions between **COMMON** and **EQUIVALENCE** statements are considered, let us treat first a situation typical of an assembly language, where the only **EQUIVALENCE** statements are of the form

EQUIVALENCE A, B+offset

where **A** and **B** are the names of locations. This statement makes **A** denote the location that is *offset* memory units beyond the location for **B**.

A sequence of **EQUIVALENCE** statements groups names into *equivalence sets* whose positions relative to one another are all defined by the **EQUIVALENCE** statements. For example, the sequence of statements

```
EQUIVALENCE A, B+100
EQUIVALENCE C, D-40
EQUIVALENCE A, C+30
EQUIVALENCE E, F
```

groups names into the sets $\{A, B, C, D\}$ and $\{E, F\}$, where E and F denote the same location. C is 70 locations after B , A is 30 after C , and D is 10 after A .

To compute the equivalence sets we create a tree for each set. Each node of a tree represents a name and contains the offset of that name relative to the name at the parent of this node. The name at the root of a tree we call the *leader*. The position of any name relative to the leader can be computed by following the path from the node for that name and adding the offsets along the way.

Example 7.12. The equivalence set $\{A, B, C, D\}$ mentioned above could be represented by the tree shown in Fig. 7.46. D is the leader, and we can discover that A is located 10 positions before D , since the sum of the offsets on the path from A to D is $100 + (-110) = -10$. $\square$

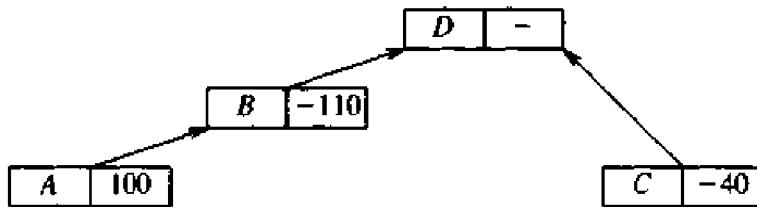


Fig. 7.46. Tree representing equivalence set.

Let us now give an algorithm for constructing trees for equivalence sets. The relevant fields in the symbol-table entries are:

1. *parent*, pointing to the symbol-table entry for the parent, null if the name is a root (or not equivalenced to anything), and
2. *offset*, giving the offset of a name relative to the parent name.

The algorithm we give assumes that any name could be the leader of an equivalence set. In practice, in an assembly language, one and only one name in the set would have an actual location defined by a pseudo-operation, and this name would be made the leader. We trust the reader can see how to modify the algorithm to make one particular name be the leader.

Algorithm 7.1. Construction of equivalence trees.

Input. A list of equivalence-defining statements of the form

EQUIVALENCE $A, B + dist$

Output. A collection of trees such that, for any name mentioned in the input list of equivalences, we may, by following the path from that name to the root and summing the *offset*'s found along the path, determine the position of the name relative to the leader.

Method. We repeat the steps of Fig. 7.47 for each equivalence statement **EQUIVALENCE** $A, B + dist$, in turn. The justification for the formula in line (12) for the offset of the leader of A relative to the leader of B is as follows. The location of A , say l_A , is equal to c plus the location of the leader of A , say m_A . The location of B , say l_B , equals d plus the location of the leader of B , say m_B . But $l_A = l_B + dist$, so $c + m_A = d + m_B + dist$. Hence $m_A - m_B$ equals $d - c + dist$. $\square$

```

begin
(1)   let  $p$  and  $q$  point to the nodes for  $A$  and  $B$ , respectively;
(2)    $c := 0$ ;  $d := 0$ ; /*  $c$  and  $d$  compute the offsets of  $A$  and  $B$ 
        from the leaders of their respective sets */
(3)   while  $parent(p) \neq \text{null}$  do begin
(4)        $c := c + offset(p)$ ;
(5)        $p := parent(p)$ 
    end; /* move  $p$  to the leader of  $a$ , accumulating
        offsets as we go */
(6)   while  $parent(q) \neq \text{null}$  do begin
(7)        $d := d + offset(q)$ ;
(8)        $q := parent(q)$ 
    end; /* do the same for  $B$  */
(9)   if  $p = q$  then /*  $A$  and  $B$  are already equivalenced */
(10)  if  $c - d \neq dist$  then error;
        /*  $A$  and  $B$  have been given two different relative positions */
    else begin /* merge the sets of  $A$  and  $B$  */
(11)       $parent(p) := q$  /* make the leader of  $A$ 
        a child of  $B$ 's leader */
(12)       $offset(p) := d - c + dist$ 
    end
end
end

```

Fig. 7.47. Equivalence algorithm.

Example 7.13. If we process

EQUIVALENCE	$A, B + 100$
EQUIVALENCE	$C, D - 40$

we get the configuration shown in Fig. 7.46, but without the offset -110 in the node for B and with no link from B to D . When we process

EQUIVALENCE	$A, C + 30$
--------------------	-------------

we find that p points to B after the while-loop of line (3) and q points to d after the while-loop of line (6). We also have $c = 100$ and $d = -40$. Then at line (11) we make D the parent of B and set the *offset* field for B to 100, which is $(-40) - (100) + 30$. $\square$

Algorithm 7.1 could take time proportional to n^2 to process n equivalences, since in the worst case the paths followed in the loops of lines (3) and (6) could include every node of their respective trees. Equivalencing requires only a tiny fraction of the time spent in compilation, so n^2 steps is not prohibitive, and an algorithm more complex than that of Fig. 7.47 is probably not justified. However, it happens that there are two easy things we can do to make Algorithm 7.1 take time that is just about linear in the number of equivalences it processes. While it is not likely that equivalence sets will be large enough, on the average, that these improvements need to be implemented, it is worth noting that equivalencing serves as a paradigm for a number of important processes involving "set merging." For example, a number of efficient algorithms for data-flow analysis depend on fast equivalence algorithms; the interested reader is referred to the bibliographic notes of Chapter 10.

The first improvement we can make is to keep a count, for each leader, of the number of nodes in its tree. Then, at lines (11) and (12), instead of arbitrarily linking the leader of A to the leader of B , link whichever has the smaller count to the other. This makes sure that the trees grow squat so paths will be short. It is left as an exercise that n equivalences performed in this manner cannot produce paths longer than $\log_2 n$ nodes.

The second idea is known as path compression. When following a path to the root in the loops of lines (3) and (6), make all nodes encountered children of the leader if they are not already so. That is, while following the path, record all the nodes $n_1, n_2, \dots, n_k$ encountered, where n_1 is the node for A or B and n_k is the leader. Then adjust offsets and make $n_1, n_2, \dots, n_{k-2}$ children of n_k by the steps in Fig. 7.48.

```

begin
   $h := \text{offset}(n_{k-1});$ 
  for  $i := k-2$  downto 1 do begin
     $\text{parent}(n_i) := n_k;$ 
     $h := h + \text{offset}(n_i);$ 
     $\text{offset}(n_i) := h$ 
  end
end
```

Fig. 7.48. Adjustment of offsets.

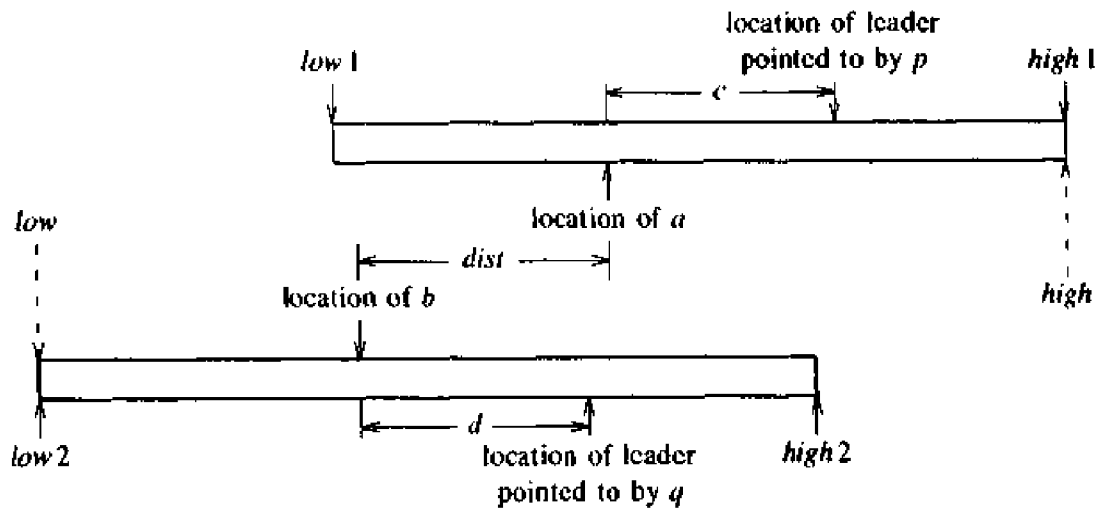
An Equivalence Algorithm for Fortran

There are several additional features that must be added to Algorithm 7.1 to make it work for Fortran. First, we must determine whether an equivalence set is in **COMMON**, which we can do by recording for each leader whether any of the names in its set are in **COMMON**, and if so, in which block.

Second, in an assembly language, one member of an equivalence set will pin down the entire set to reality by being a label of a statement, thus allowing the addresses denoted by all names in the set to be computed relative to that one location. In Fortran, however, it is the compiler's job to determine storage locations, so an equivalence set not in **COMMON** may be viewed as "floating" until the compiler determines the position of the whole set in its appropriate data area. To do so correctly, the compiler needs to know the extent of the equivalence set, that is, the number of locations which the names in the set collectively occupy. To handle this problem we attach to the leader two fields, *low* and *high*, giving the offsets relative to the leader of the lowest and highest locations used by any member of the equivalence set. Third, there are minor problems introduced by the fact that names can be arrays and locations in the middle of an array can be equivalenced to locations in other arrays.

Since there are three fields (*low*, *high*, and a pointer to a **COMMON** block) that must be associated with each leader, we do not want to allocate space for these fields in all symbol-table entries. One course of action is to use the *parent* field from Algorithm 7.1 to point, in the case of the leader, to a record in a new table with three fields, *low*, *high*, and *comblk*. As this table and the symbol table occupy disjoint areas, we can tell to which table a pointer points. Alternatively, the symbol table can contain a bit indicating whether a name is currently a leader. If space is really at a premium, an alternative algorithm avoiding this extra table at the cost of a bit more programming effort is discussed in the exercises.

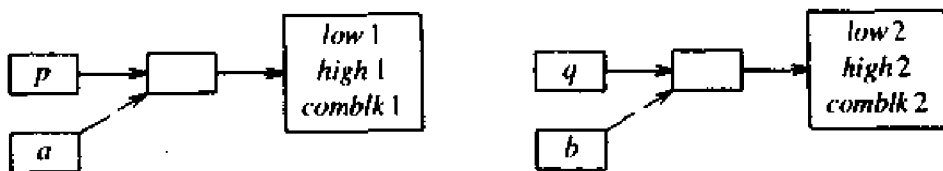
Let us consider the calculation that must replace lines (11) and (12) of Fig. 7.47. The situation in which two equivalence sets, whose leaders are pointed to by p and q , must be merged is depicted in Fig. 7.49(a). The data structure representing the two sets appears in Fig. 7.49(b). First we, must check that there are not two members among the two equivalence sets that are in **COMMON**. Even if both are in the same block, the Fortran standard forbids their being equivalenced. If any one **COMMON** block contains a member of either equivalence set, then the merged set has a pointer to the record for that block in *comblk*. The code doing this check, assuming the leader pointed to by q becomes the leader of the merged set, is shown in Fig. 7.50. In place of lines (11) and (12) of Fig. 7.47 we must also compute the extent of the merged equivalence set. Figure 7.49(a) indicates the formulas for the new values of *low* and *high* relative to the leader pointed to by q .



$$low = \min(low\ 2, low\ 1 - c + dist + d)$$

$$high = \max(high\ 2, high\ 1 - c + dist + d)$$

(a) relative positions of equivalence sets



(b) data structure

Fig. 7.49. Merging equivalence sets.

```

begin
  comblk 1 := comblk(parent(p));
  comblk 2 := comblk(parent(q));
  if comblk 1 ≠ null and comblk 2 ≠ null then
    error; /* two names in COMMON equivalent */
  else if comblk 2 = null then
    comblk(parent(q)) := comblk 1
end

```

Fig. 7.50. Computing COMMON blocks.

Thus we must do

```

begin
    low(parent(q)) := min(low(parent(q)), low(parent(p)) - c + dist + d);
    high(parent(q)) := max(high(parent(q)), high(parent(p)) - c + dist + d)
end

```

These statements are followed by lines (11) and (12) of Fig. 7.47 to effect the merger of the two equivalence sets.

Two last details must be covered to make Algorithm 7.1 work for Fortran. In Fortran, we may equivalence positions in the middle of arrays to other positions in other arrays or to simple names. The offset of an array *A* from its leader means the offset of the first location of *A* from the first location of the leader. If a location like *A*(5, 7) is equivalenced to, say, *B*(20), we must compute the position of *A*(5, 7) relative to *A*(1, 1) and initialize *c* to the negative of this distance in line (2) of Fig. 7.47. Similarly, *d* must be initialized to the negative of the position of *B*(20) relative to *B*(1). The formulas in Section 8.3 together with a knowledge of the size of elements of the arrays *A* and *B*, are sufficient to calculate the initial values of *c* and *d*.

The last detail to be covered is the fact that Fortran allows an **EQUIVALENCE** which involves many locations, such as

```
EQUIVALENCE (A(5,7), B(20), C, D(4,5,6))
```

These may be treated as

```

EQUIVALENCE (B(20), A(5,7))
EQUIVALENCE (C, A(5,7))
EQUIVALENCE (D(4,5,6), A(5,7))

```

Note that if we do the equivalences in this order, only *A* becomes the leader of a set of more than one element. A record with *low*, *high*, and *comblk* can be used many times for "equivalence sets" of a single name.

Mapping Data Areas

We may now describe the rules whereby space in the various data areas is assigned for each routine's names.

1. For each **COMMON** block, visit all names declared to be in that block in the order of their declarations (use the chains of **COMMON** names created in the symbol table for this purpose). Allocate the number of words needed for each name in turn, keeping a count of the number of words allocated, so offsets can be computed for each name. If a name *A* is equivalenced, the extent of its equivalence set does not matter, but we must check that the *low* value for the leader of *A* does not extend past the beginning of the **COMMON** block. Consult the *high* value for the leader to put a lower limit on the last word of the block. We leave the exact formulas for these calculations to the reader.

2. Visit all names for the routine in any order.
 - a) If a name is in **COMMON**, do nothing. Space has been allocated in (1).
 - b) If a name is not in **COMMON** and not equivalenced, allocate the necessary number of words in the data area for the routine.
 - c) If a name *A* is equivalenced, find its leader, say *L*. If *L* has already been given a position in the data area for the routine, compute the position of *A* by adding to that position all the *offset*'s found in the path from *A* to *L* in the tree representing the equivalence set of *A* and *L*. If *L* has not been given a position, allocate the next *high-low* words in the data area for the equivalence set. The position of *L* in these words is *-low* words from the beginning, and the position of *A* can be calculated by summing *offset*'s as before.

EXERCISES

- 7.1 Using the scope rules of Pascal, determine the declarations that apply to each occurrence of the names *a* and *b* in Fig. 7.51. The output of the program consists of the integers 1 through 4.

```

program a(input, output);
procedure b(u,v,x,y: integer);
  var  a : record a, b : integer end;
       b : record b, a : integer end;
begin
  with a do begin a := u; b := v end;
  with b do begin a := x; b := y end;
  writeln(a.a, a.b, b.a, b.b)
end;
begin
  b(1, 2, 3, 4)
end.

```

Fig. 7.51. Pascal program with several declarations of *a* and *b*.

- 7.2 Consider a block-structured language in which a name can be declared to be an integer or a real. Suppose that expressions are represented by a terminal *expr* and that the only statements are assignments, conditionals, *whiles*, and sequences of statements. Assuming that integers are allocated one word and that reals are allocated two words, give a syntax-directed algorithm (based on a reasonable grammar for declarations and blocks) to determine bindings from names to words that can be used by an activation of a block. Does your

allocation use the minimum number of words adequate for any execution of the block?

- *7.3 In Section 7.4 we claimed that the display could be maintained correctly if each procedure at depth i stored $d[i]$ at the beginning of an activation and restored $d[i]$ at the end. Prove by induction on the number of calls that each procedure sees a correct display.
- 7.4 A *macro* is a form of procedure, implemented by literally substituting the body for each procedure call. Figure 7.52 shows a Pic program and its output. The first two lines define the macros `show` and `small`. The bodies of the macros are contained between the two `%` signs on the lines. Each of the four circles in the figure is drawn using `show`; the radius of the circle is given by the nonlocal name `r`. Blocks in Pic are delimited by `[` and `]`. Each variable assigned to within a block is implicitly declared within the block. From the output, what can you say about the scope of each occurrence of `r`?

```
define show % { circle radius r at Here } %
define small % [ r = 1/12; show ] %
[
    r = 1/6;
    show; small;
    move;
    show; small;
]
```



Fig. 7.52. Circles drawn by a Pic program.

- 7.5 Write a procedure to insert an item into a linked list by passing a pointer to the head of the list. Under what parameter passing mechanisms does this procedure work?
- 7.6 What is printed by the program in Fig. 7.53, assuming (a) call-by-value, (b) call-by-reference, (c) copy-restore linkage, (d) call-by-name?
- 7.7 When a procedure is passed as a parameter in a lexically scoped language, its nonlocal environment can be passed using an access link. Give an algorithm to determine this link.

```

program main(input, output);
  procedure p(x, y, z);
    begin
      y := y + 1;
      z := z + x;
    end;
  begin
    a := 2;
    b := 3;
    p(a + b, a, a);
    print a
  end.

```

Fig. 7.53. Pseudo-program illustrating parameter passing.

7.8 The three kinds of environments that could be associated with a procedure passed as a parameter are illustrated by the Pascal program in Fig. 7.54. The *lexical*, *passing*, and *activation* environments of such a procedure consist of the bindings of identifiers at the point at which the procedure is defined, passed as a parameter, and activated, respectively. Consider function *f*, passed as a parameter on line 11.

```

(1) program param(input, output);
(2)   procedure b(function h(n: integer): integer);
(3)     var m : integer;
(4)     begin m := 3; writeln(h(2)) end { b };
(5)   procedure c;
(6)     var m : integer;
(7)     function f(n : integer) : integer;
(8)       begin f := m + n end { f };
(9)     procedure r;
(10)       var m : integer;
(11)       begin m := 7; b(f) end { r };
(12)     begin m := 0; r end { c };
(13)   begin
(14)     c
(15)   end.

```

Fig. 7.54. Example of lexical, passing, and activation environments.

Using the lexical, passing, and activation environments for f , nonlocal m on line 8 is in the scope of the declarations of m on lines 6, 10, and 3, respectively.

- a) Draw the activation tree for this program.
- b) What is the output of the program, using the lexical, passing, and activation environments for f ?
- *c) Modify the display implementation of a lexically scoped language to set up the lexical environment correctly when a procedure passed as a parameter is activated.

***7.9** The statement $f := a$ on line 11 of the pseudo-program in Fig. 7.55 calls function a , which passes function $addm$ back as a result.

- a) Draw the activation tree for an execution of this program.
- b) Suppose that lexical scope is used for nonlocal names. Why will the program fail if stack allocation is used?
- c) What is the output of the program with heap allocation?

```

(1) program ret(input, output);
(2) var  $f$  : function (integer) : integer;

(3) function  $a$  : function (integer) : integer;
(4)   var  $m$  : integer;
(5)   function  $addm$ ( $n$  : integer) : integer;
(6)     begin return  $m + n$  end;
(7)   begin  $m := 0$ ; return  $addm$  end;

(8) procedure  $b$ ( $g$  : function (integer) : integer);
(9)   begin writeln( $g(2)$ ) end;

(10) begin
(11)    $f := a$ ;  $b(f)$ 
(12) end.
```

Fig. 7.55. Pseudo-program in which function $addm$ is returned as a result.

***7.10** Certain languages, like Lisp, have the ability to return newly created procedures at run time. In Fig. 7.56, all functions, whether defined in the source text or created at run time, take at most one argument and return one value, either a function or a real. The operator $\circ$ stands for composition of functions; that is $(f \circ g)(x) = f(g(x))$.

- a) What value is printed by *main*?
- *b) Suppose that whenever a procedure p is created and returned, its activation record becomes a child of the activation record of the function returning p . The passing environment of p can then be

maintained by keeping a tree of activation records rather than a stack. What is the tree of activation records when a is computed by *main* in Fig. 7.56?

- *c) Alternatively, suppose an activation record for p is created when p is activated, and made a child of the activation record for the procedure calling p . This approach can be used to maintain the activation environment for p . Draw snapshots of the activation records and their parent-child relationships as the statements in *main* are executed. Is a stack sufficient to hold activation records when this approach is used?

```

function  $f(x : \text{function})$ ;
  var  $y : \text{function}$ ;
       $y := x \circ h$ ; /* creates  $y$  when executed */
  return  $y$ 
end {  $f$  };

function  $h()$ ;
  return  $\sin$ 
end {  $h$  };

function  $g(z : \text{function})$ ;
  var  $w : \text{function}$ ;
       $w := \arctan \circ z$ ; /* creates  $w$  when executed */
  return  $w$ 
end {  $g$  };

function  $\text{main}()$ ;
  var  $a : \text{real}$ ;
       $u, v : \text{function}$ ;
       $v := f(g)$ ;
       $u := v()$ ;
       $a := u(\pi/2)$ ;
      print  $a$ 
end {  $\text{main}$  }.

```

Fig. 7.56. Pseudo-program creating functions at run time.

- 7.11 Another way to handle deletion from hash tables for names whose scope has been passed (as in Section 7.6) is to leave expired names on a list until that list is again searched. Assuming entries include the name of the procedure in which the declaration is made, we can in principle tell whether a name is old, and delete it if so. Give an indexing scheme for procedures that enables us to tell in $O(1)$ time whether a procedure is "old," i.e., its scope has been passed.

- 7.12** Many hash functions can be characterized by a sequence of integer constants $\alpha_0, \alpha_1, \dots$. If c_i , $1 \leq i \leq n$, is the integer value of the i th character in string s , then the string is hashed to

$$\text{hash}(s) = (\alpha_0 + \sum_{i=1}^n \alpha_i c_i) \bmod m$$

where m is the size of the hash table. For each of the following cases, determine the sequence of constants $\alpha_0, \alpha_1, \dots$ or show that no such sequence exists. Each case determines an integer; a hash value is obtained by taking that integer mod m .

- Take the sum of the characters.
- Take the sum of the first and last characters.
- Take h_n , where $h_0 = 0$ and $h_i = 2h_{i-1} + c_i$.
- Treat the bits in the middle 4 characters as a 32-bit integer.
- A 32-bit integer can be viewed as consisting of 4 bytes, where each byte is a digit that takes on 256 possible values. Starting with 0000, for $1 \leq i \leq n$, add c_i into byte $i \bmod 4$, with carries permitted. That is, c_1 and c_5 are added into byte 1, c_2 and c_6 into byte 2, and so on. Return the final value.

- *7.13** Why do hash functions characterized by a sequence of integers $\alpha_0, \alpha_1, \dots$ as in Exercise 7.12 sometimes perform poorly if the input consists of consecutive strings, e.g., $v000, v001, \dots$? The symptom is that somewhere along the way, their behavior deviates from random and can be predicted.

- **7.14** When n strings are hashed into m lists, the mean number of strings per list is n/m , no matter how unevenly the strings are distributed. Suppose that d is a "distribution," i.e., a random string is placed on the i th list with probability $d(i)$. Suppose that a hash function with distribution d happens to place b_j randomly selected strings in list j , $0 \leq j \leq m-1$. Show that the expected value $W = \sum_{j=0}^{m-1} (b_j)(b_j+1)/2$ is linearly related to the variance of the distribution d . For a uniform distribution show that the expected value of W is $(n/2m)(n+2m-1)$.

- 7.15** Suppose we have the following sequence of declarations in a Fortran program.

```
SUBROUTINE SUB(X,Y)
  INTEGER A, B(20), C(10,15), D, E
  COMPLEX F, G
  COMMON /CBLK/ D, E
  EQUIVALENCE (G, B(2))
  EQUIVALENCE (D, F, B(1))
```

Show the contents of the data areas for SUB and CBLK (at least the portion of CBLK's area accessible from SUB). Why is there no space for X and Y?

***7.16** A useful data structure for equivalence computations is the *ring structure*. We use one pointer and an offset field in each symbol-table entry to link members of an equivalence set. This structure is suggested in Fig. 7.57, where A, B, C, and D are equivalent, and E and F are equivalent, with the location of B being 20 words after that of A, and so on.

- Give an algorithm to compute the offset of X relative to Y, assuming that X and Y are in the same equivalence set.
- Give an algorithm to compute *low* and *high*, as defined in Section 7.9, relative to the location of some name Z.
- Give an algorithm to process

EQUIVALENCE U, V

Do not assume that U and V are necessarily in different equivalence sets.

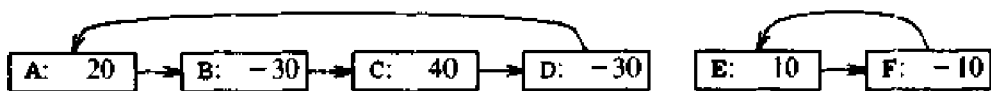


Fig. 7.57. Ring structures.

***7.17** The algorithm to map data areas given in Section 7.9 requires that we verify that *low* for the leader of A's equivalence set does not cause the space for the equivalence set of A to extend before the beginning of the COMMON block and that we calculate *high* for the leader of A to increase the upper limit of the COMMON block, if necessary. Give formulas in terms of *next*, the offset of A in the COMMON block, and *last*, the current last word of the block, to make the test and to update *last*, if necessary.

BIBLIOGRAPHIC NOTES

Stacks have played an essential role in the implementation of recursive functions. McCarthy [1981, p. 178] recalls that during a Lisp-implementation project begun in 1958 it was decided to "use a single contiguous public stack array to save the values of variables and subroutine return addresses in the implementation of recursive subroutines." The inclusion of blocks and recursive procedures in Algol 60 — see Naur [1981, Section 2.10] for a detailed account of their design — also stimulated development of stack allocation. The idea of a display for accessing nonlocals in a lexically scoped language is due to Dijkstra [1960, 1963]. Although Lisp uses dynamic scope, it is possible to achieve the effect of lexical scope using "funargs" consisting of a function and an access link; McCarthy [1981] describes the development of this

mechanism. Successors of Lisp such as Common Lisp (Steele [1984]) have moved away from dynamic scope.

Explanations of bindings for names can be found in textbooks on programming languages, see for example Abelson and Sussman [1985], Pratt [1984], or Tennent [1981]. An alternative approach, suggested in Chapter 2, is to read the description of a compiler. The step-by-step development in Kernighan and Pike [1984] starts with a calculator for arithmetic expressions and builds an interpreter for a simple language with recursive procedures. Or see the code for Pascal-S in Wirth [1981]. A detailed description of stack allocation, the use of a display, and dynamic allocation of arrays appears in Randell and Russell [1964].

Johnson and Ritchie [1981] discuss the design of a calling sequence that allows the number of arguments of a procedure to vary from call to call. A general method for setting a global display is to follow the chain of access links, setting the display elements in the process. The approach of Section 7.4 that touches just one element, seems to have been “well known” for some time; a published reference is Rohl [1975]. Moses [1970] discusses the distinctions between the environments that apply when a function is passed as a parameter and considers the problems that arise when such environments are implemented using shallow and deep access. Stack allocation cannot be used for languages with coroutines or multiple processes. Lampson [1982] considers fast implementations using heap allocation.

In mathematical logic, quantified variables of limited scope and substitution appear with the *Begriffsschrift* of Frege [1879]. Substitution and parameter passing have been the subject of much debate in both the mathematical logic and programming language communities. Church [1956, p. 288] observes, “Especially difficult is the matter of correct statement of the rule of substitution for functional variables,” and relates the development of such a rule for the propositional calculus. The lambda calculus of Church [1941] has been applied to environments in programming languages, for example by Landin [1964]. A pair consisting of a function and an access link is often referred to as a *closure*, following Landin [1964].

Data structures for symbol tables and algorithms for searching them are discussed in detail in Knuth [1973b] and Aho, Hopcroft, and Ullman [1974, 1983]. The lore of hashing is treated in Knuth [1973b] and Morris [1968b]. The original paper discussing hashing is Peterson [1957]. More on symbol-table organization techniques can be found in McKeeman [1976]. Example 7.10 is from Bentley, Cleveland, and Sethi [1985]. Reiss [1983] describes a symbol-table generator.

Equivalence algorithms have been described by Arden, Galler, and Graham [1961] and Galler and Fischer [1964]; we have adopted the latter approach. The efficiency of equivalence algorithms is discussed in Fischer [1972], Hopcroft and Ullman [1973], and Tarjan [1975].

Intermediate Code Generation

In the analysis-synthesis model of a compiler, the front end translates a source program into an intermediate representation from which the back end generates target code. Details of the target language are confined to the back end, as far as possible. Although a source program can be translated directly into the target language, some benefits of using a machine-independent intermediate form are:

1. Retargeting is facilitated; a compiler for a different machine can be created by attaching a back end for the new machine to an existing front end.
2. A machine-independent code optimizer can be applied to the intermediate representation. Such optimizers are discussed in detail in Chapter 10.

This chapter shows how the syntax-directed methods of Chapters 2 and 5 can be used to translate into an intermediate form programming language constructs such as declarations, assignments, and flow-of-control statements. For simplicity, we assume that the source program has already been parsed and statically checked, as in the organization of Fig. 8.1. Most of the syntax-directed definitions in this chapter can be implemented during either bottom-up or top-down parsing using the techniques of Chapter 5, so intermediate code generation can be folded into parsing, if desired.

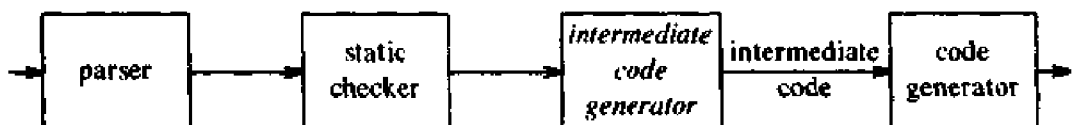


Fig. 8.1. Position of intermediate code generator.

8.1 INTERMEDIATE LANGUAGES

Syntax trees and postfix notation, introduced in Sections 5.2 and 2.3, respectively, are two kinds of intermediate representations. A third, called three-address code, will be used in this chapter. The semantic rules for generating three-address code from common programming language constructs are similar to those for constructing syntax trees or for generating postfix notation.

Graphical Representations

A syntax tree depicts the natural hierarchical structure of a source program. A dag gives the same information but in a more compact way because common subexpressions are identified. A syntax tree and dag for the assignment statement $a := b * -c + b * -c$ appear in Fig. 8.2.

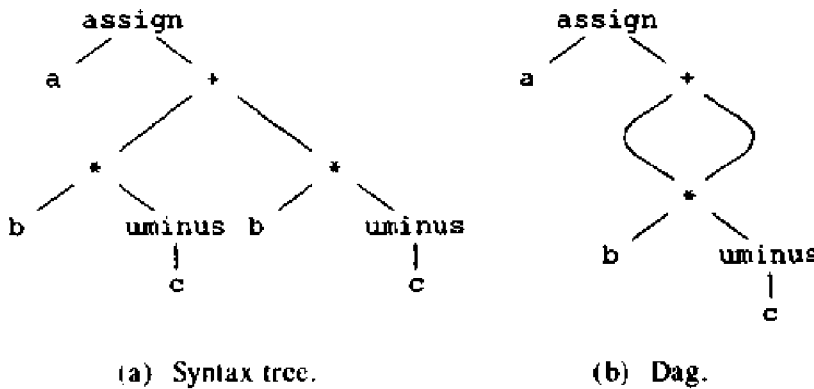


Fig. 8.2. Graphical representations of $a := b * -c + b * -c$.

Postfix notation is a linearized representation of a syntax tree; it is a list of the nodes of the tree in which a node appears immediately after its children. The postfix notation for the syntax tree in Fig. 8.2(a) is

$$a \ b \ c \ \text{uminus} \ * \ b \ c \ \text{uminus} \ * \ + \ \text{assign} \quad (8.1)$$

The edges in a syntax tree do not appear explicitly in postfix notation. They can be recovered from the order in which the nodes appear and the number of operands that the operator at a node expects. The recovery of edges is similar to the evaluation, using a stack, of an expression in postfix notation. See Section 2.8 for more details and the relationship between postfix notation and code for a stack machine.

Syntax trees for assignment statements are produced by the syntax-directed definition in Fig. 8.3; it is an extension of one in Section 5.2. Nonterminal S generates an assignment statement. The two binary operators $+$ and $*$ are examples of the full operator set in a typical language. Operator associativities and precedences are the usual ones, even though they have not been put into the grammar. This definition constructs the tree of Fig. 8.2(a) from the input $a := b * -c + b * -c$.

PRODUCTION	SEMANTIC RULE
$S \rightarrow id := E$	$S.nptr := mknode('assign', mkleaf(id, id.place), E.nptr)$
$E \rightarrow E_1 + E_2$	$E.nptr := mknode('+', E_1.nptr, E_2.nptr)$
$E \rightarrow E_1 * E_2$	$E.nptr := mknode('*', E_1.nptr, E_2.nptr)$
$E \rightarrow - E_1$	$E.nptr := mkunode('uminus', E_1.nptr)$
$E \rightarrow (E_1)$	$E.nptr := E_1.nptr$
$E \rightarrow id$	$E.nptr := mkleaf(id, id.place)$

Fig. 8.3. Syntax-directed definition to produce syntax trees for assignment statements.

This same syntax-directed definition will produce the dag in Fig. 8.2(b) if the functions *mkunode*(*op*, *child*) and *mknode*(*op*, *left*, *right*) return a pointer to an existing node whenever possible, instead of constructing new nodes. The token *id* has an attribute *place* that points to the symbol-table entry for the identifier. In Section 8.3, we show how a symbol-table entry can be found from an attribute *id.name*, representing the lexeme associated with that occurrence of *id*. If the lexical analyzer holds all lexemes in a single array of characters, then attribute *name* might be the index of the first character of the lexeme.

Two representations of the syntax tree in Fig. 8.2(a) appear in Fig. 8.4. Each node is represented as a record with a field for its operator and additional fields for pointers to its children. In Fig 8.4(b), nodes are allocated from an array of records and the index or position of the node serves as the pointer to the node. All the nodes in the syntax tree can be visited by following pointers, starting from the root at position 10.

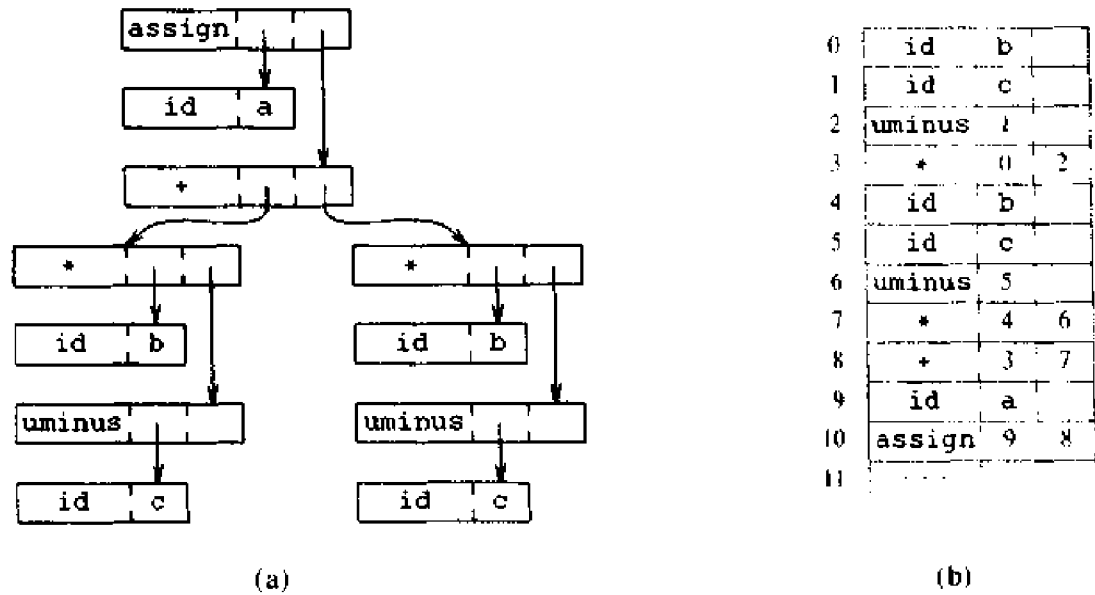


Fig. 8.4. Two representations of the syntax tree in Fig. 8.2(a).

Three-Address Code

Three-address code is a sequence of statements of the general form

$$x := y \text{ op } z$$

where x , y , and z are names, constants, or compiler-generated temporaries; op stands for any operator, such as a fixed- or floating-point arithmetic operator, or a logical operator on boolean-valued data. Note that no built-up arithmetic expressions are permitted, as there is only one operator on the right side of a statement. Thus a source language expression like $x + y * z$ might be translated into a sequence

$$\begin{aligned} t_1 &:= y * z \\ t_2 &:= x + t_1 \end{aligned}$$

where t_1 and t_2 are compiler-generated temporary names. This unraveling of complicated arithmetic expressions and of nested flow-of-control statements makes three-address code desirable for target code generation and optimization. (See Chapters 10 and 12.) The use of names for the intermediate values computed by a program allows three-address code to be easily rearranged — unlike postfix notation.

Three-address code is a linearized representation of a syntax tree or a dag in which explicit names correspond to the interior nodes of the graph. The syntax tree and dag in Fig. 8.2 are represented by the three-address code sequences in Fig. 8.5. Variable names can appear directly in three-address statements, so Fig. 8.5(a) has no statements corresponding to the leaves in Fig. 8.4.

$$\begin{aligned} t_1 &:= -c \\ t_2 &:= b * t_1 \\ t_3 &:= -c \\ t_4 &:= b * t_3 \\ t_5 &:= t_2 + t_4 \\ a &:= t_5 \end{aligned}$$

(a) Code for the syntax tree.

$$\begin{aligned} t_1 &:= -c \\ t_2 &:= b * t_1 \\ t_5 &:= t_2 + t_2 \\ a &:= t_5 \end{aligned}$$

(b) Code for the dag.

Fig. 8.5. Three-address code corresponding to the tree and dag in Fig. 8.2.

The reason for the term “three-address code” is that each statement usually contains three addresses, two for the operands and one for the result. In the implementations of three-address code given later in this section, a programmer-defined name is replaced by a pointer to a symbol-table entry for that name.

Types of Three-Address Statements

Three-address statements are akin to assembly code. Statements can have symbolic labels and there are statements for flow of control. A symbolic label represents the index of a three-address statement in the array holding intermediate code. Actual indices can be substituted for the labels either by making a separate pass, or by using "backpatching," discussed in Section 8.6.

Here are the common three-address statements used in the remainder of this book:

1. Assignment statements of the form $x := y \text{ op } z$, where op is a binary arithmetic or logical operation.
2. Assignment instructions of the form $x := op \ y$, where op is a unary operation. Essential unary operations include unary minus, logical negation, shift operators, and conversion operators that, for example, convert a fixed-point number to a floating-point number.
3. *Copy statements* of the form $x := y$ where the value of y is assigned to x .
4. The unconditional jump `goto L`. The three-address statement with label L is the next to be executed.
5. Conditional jumps such as `if x relop y goto L`. This instruction applies a relational operator ($<$, $=$, $>$, etc.) to x and y , and executes the statement with label L next if x stands in relation *relop* to y . If not, the three-address statement following `if x relop y goto L` is executed next, as in the usual sequence.
6. `param x` and `call p, n` for procedure calls and `return y`, where y representing a returned value is optional. Their typical use is as the sequence of three-address statements

```

param x1
param x2
...
param xn
call p, n

```

generated as part of a call of the procedure $p(x_1, x_2, \dots, x_n)$. The integer n indicating the number of actual-parameters in "`call p, n`" is not redundant because calls can be nested. The implementation of procedure calls is outlined in Section 8.7.

7. Indexed assignments of the form $x := y[i]$ and $x[i] := y$. The first of these sets x to the value in the location i memory units beyond location y . The statement $x[i] := y$ sets the contents of the location i units beyond x to the value of y . In both these instructions, x , y , and i refer to data objects.
8. Address and pointer assignments of the form $x := \&y$, $x := *y$, and

$*x := y$. The first of these sets the value of x to be the location of y . Presumably y is a name, perhaps a temporary, that denotes an expression with an l -value such as $A[i, j]$, and x is a pointer name or temporary. That is, the r -value of x is the l -value (location) of some object. In the statement $x := *y$, presumably y is a pointer or a temporary whose r -value is a location. The r -value of x is made equal to the contents of that location. Finally, $*x := y$ sets the r -value of the object pointed to by x to the r -value of y .

The choice of allowable operators is an important issue in the design of an intermediate form. The operator set must clearly be rich enough to implement the operations in the source language. A small operator set is easier to implement on a new target machine. However, a restricted instruction set may force the front end to generate long sequences of statements for some source language operations. The optimizer and code generator may then have to work harder if good code is to be generated.

Syntax-Directed Translation into Three-Address Code

When three-address code is generated, temporary names are made up for the interior nodes of a syntax tree. The value of nonterminal E on the left side of $E \rightarrow E_1 + E_2$ will be computed into a new temporary t . In general, the three-address code for $id := E$ consists of code to evaluate E into some temporary t , followed by the assignment $id.place := t$. If an expression is a single identifier, say y , then y itself holds the value of the expression. For the moment, we create a new name every time a temporary is needed; techniques for reusing temporaries are given in Section 8.3.

The S-attributed definition in Fig. 8.6 generates three-address code for assignment statements. Given input $a := b * -c + b * -c$, it produces the code in Fig. 8.5(a). The synthesized attribute $S.code$ represents the three-address code for the assignment S . The nonterminal E has two attributes:

1. $E.place$, the name that will hold the value of E , and
2. $E.code$, the sequence of three-address statements evaluating E .

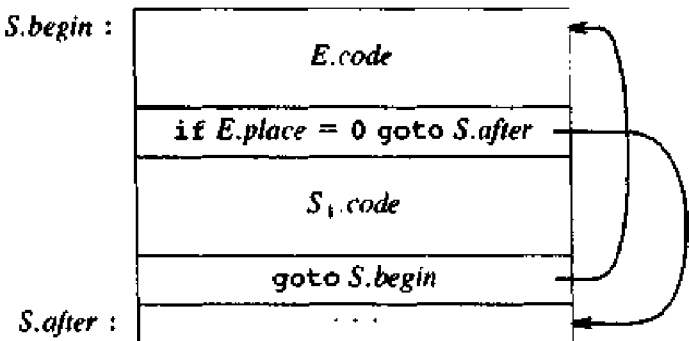
The function *newtemp* returns a sequence of distinct names $t_1, t_2, \dots$ in response to successive calls.

For convenience, we use the notation $gen(x := y + z)$ in Fig. 8.6 to represent the three-address statement $x := y + z$. Expressions appearing instead of variables like x , y , and z are evaluated when passed to *gen*, and quoted operators or operands, like $+$, are taken literally. In practice, three-address statements might be sent to an output file, rather than built up into the *code* attributes.

Flow-of-control statements can be added to the language of assignments in Fig. 8.6 by productions and semantic rules like the ones for while statements in Fig. 8.7. In the figure, the code for $S \rightarrow \text{while } E \text{ do } S_1$ is generated using new attributes $S.begin$ and $S.after$ to mark the first statement in the code for E

PRODUCTION	SEMANTIC RULES
$S \rightarrow id := E$	$S.code := E.code \parallel gen(id.place := E.place)$
$E \rightarrow E_1 + E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel$ $gen(E.place := E_1.place + E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel$ $gen(E.place := E_1.place * E_2.place)$
$E \rightarrow - E_1$	$E.place := newtemp;$ $E.code := E_1.code \parallel gen(E.place := 'uminus' E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$ $E.code := E_1.code$
$E \rightarrow id$	$E.place := id.place;$ $E.code := ''$

Fig. 8.6. Syntax-directed definition to produce three-address code for assignments.



PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{while } E \text{ do } S_1$	$S.begin := newlabel;$ $S.after := newlabel;$ $S.code := gen(S.begin ':') \parallel$ $E.code \parallel$ $gen('if' E.place '=' '0' 'goto' S.after) \parallel$ $S_1.code \parallel$ $gen('goto' S.begin) \parallel$ $gen(S.after ':')$

Fig. 8.7. Semantic rules generating code for a while statement.

and the statement following the code for S , respectively. These attributes represent labels created by a function *newlabel* that returns a new label every time it is called. Note that $S.after$ becomes the label of the statement that comes after the code for the while statement. We assume that a non-zero expression represents true; that is, when the value of E becomes zero, control leaves the while statement.

Expressions that govern the flow of control may in general be boolean expressions containing relational and logical operators. The semantic rules for while statements in Section 8.6 differ from those in Fig. 8.7 to allow for flow of control within boolean expressions.

Postfix notation can be obtained by adapting the semantic rules in Fig. 8.6 (or see Fig. 2.5). The postfix notation for an identifier is the identifier itself. The rules for the other productions concatenate only the operator after the code for the operands. For example, associated with the production $E \rightarrow -E_1$ is the semantic rule

$$E.code := E_1.code \parallel 'uminus'$$

In general, the intermediate form produced by the syntax-directed translations in this chapter can be changed by making similar modifications to the semantic rules.

Implementations of Three-Address Statements

A three-address statement is an abstract form of intermediate code. In a compiler, these statements can be implemented as records with fields for the operator and the operands. Three such representations are quadruples, triples, and indirect triples.

Quadruples

A quadruple is a record structure with four fields, which we call *op*, *arg1*, *arg2*, and *result*. The *op* field contains an internal code for the operator. The three-address statement $x := y \text{ op } z$ is represented by placing y in *arg1*, z in *arg2*, and x in *result*. Statements with unary operators like $x := -y$ or $x := y$ do not use *arg2*. Operators like *param* use neither *arg2* nor *result*. Conditional and unconditional jumps put the target label in *result*. The quadruples in Fig. 8.8(a) are for the assignment $a := b + -c + b + -c$. They are obtained from the three-address code in Fig. 8.5(a).

The contents of fields *arg1*, *arg2*, and *result* are normally pointers to the symbol-table entries for the names represented by these fields. If so, temporary names must be entered into the symbol table as they are created.

Triples

To avoid entering temporary names into the symbol table, we might refer to a temporary value by the position of the statement that computes it. If we do

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>	<i>result</i>
(0)	uminus	c		t ₁
(1)	*	b	t ₁	t ₂
(2)	uminus	c		t ₃
(3)	*	b	t ₃	t ₄
(4)	+	t ₂	t ₄	t ₅
(5)	:=	t ₅		a

(a) Quadruples

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

(b) Triples

Fig. 8.8. Quadruple and triple representations of three-address statements.

so, three-address statements can be represented by records with only three fields: *op*, *arg 1* and *arg 2*, as in Fig. 8.8(b). The fields *arg 1* and *arg 2*, for the arguments of *op*, are either pointers to the symbol table (for programmer-defined names or constants) or pointers into the triple structure (for temporary values). Since three fields are used, this intermediate code format is known as triples.¹ Except for the treatment of programmer-defined names, triples correspond to the representation of a syntax tree or dag by an array of nodes, as in Fig. 8.4.

Parenthesized numbers represent pointers into the triple structure, while symbol-table pointers are represented by the names themselves. In practice, the information needed to interpret the different kinds of entries in the *arg 1* and *arg 2* fields can be encoded into the *op* field or some additional fields. The triples in Fig. 8.8(b) correspond to the quadruples in Fig. 8.8(a). Note that the copy statement *a := t₅* is encoded in the triple representation by placing *a* in the *arg 1* field and using the operator *assign*.

A ternary operation like *x[i] := y* requires two entries in the triple structure, as shown in Fig. 8.9(a), while *x := y[i]* is naturally represented as two operations in Fig. 8.9(b).

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(0)	[] =	x	i
(1)	assign	(0)	y

(a) *x[i] := y*

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(0)	= []	y	i
(1)	assign	x	(0)

(b) *x := y[i]*

Fig. 8.9. More triple representations.

¹ Some refer to triples as “two-address code,” preferring to identify “quadruples” with the term “three-address code.” We shall, however, treat “three-address code” as an abstract notion with various implementations, triples and quadruples being the principal ones.

Indirect Triples

Another implementation of three-address code that has been considered is that of listing pointers to triples, rather than listing the triples themselves. This implementation is naturally called indirect triples.

For example, let us use an array *statement* to list pointers to triples in the desired order. Then the triples in Fig. 8.8(b) might be represented as in Fig. 8.10.

	<i>statement</i>		<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(0)	(14)	(14)	uminus	c	
(1)	(15)	(15)	*	b	(14)
(2)	(16)	(16)	uminus	c	
(3)	(17)	(17)	*	b	(16)
(4)	(18)	(18)	+	(15)	(17)
(5)	(19)	(19)	assign	a	(18)

Fig. 8.10. Indirect triples representation of three-address statements.

Comparison of Representations: The Use of Indirection

The difference between triples and quadruples may be regarded as a matter of how much indirection is present in the representation. When we ultimately produce target code, each name, temporary or programmer-defined, will be assigned some run-time memory location. This location will be placed in the symbol-table entry for the datum. Using the quadruple notation, a three-address statement defining or using a temporary can immediately access the location for that temporary via the symbol table.

A more important benefit of quadruples appears in an optimizing compiler, where statements are often moved around. Using the quadruple notation, the symbol table interposes an extra degree of indirection between the computation of a value and its use. If we move a statement computing *x*, the statements using *x* require no change. However, in the triples notation, moving a statement that defines a temporary value requires us to change all references to that statement in the *arg 1* and *arg 2* arrays. This problem makes triples difficult to use in an optimizing compiler.

Indirect triples present no such problem. A statement can be moved by reordering the *statement* list. Since pointers to temporary values refer to the *op-arg 1-arg 2* array(s), which are not changed, none of those pointers need be changed. Thus, indirect triples look very much like quadruples as far as their utility is concerned. The two notations require about the same amount of space and they are equally efficient for reordering of code. As with ordinary triples, allocation of storage to those temporaries needing it must be deferred to the code generation phase. However, indirect triples can save some space

compared with quadruples if the same temporary value is used more than once. The reason is that two or more entries in the *statement* array can point to the same line of the *op-arg 1-arg 2* structure. For example, lines (14) and (16) of Fig. 8.10 could be combined and we could then combine (15) and (17).

8.2 DECLARATIONS

As the sequence of declarations in a procedure or block is examined, we can lay out storage for names local to the procedure. For each local name, we create a symbol-table entry with information like the type and the relative address of the storage for the name. The relative address consists of an offset from the base of the static data area or the field for local data in an activation record.

When the front end generates addresses, it may have a target machine in mind. Suppose that addresses of consecutive integers differ by 4 on a byte-addressable machine. The address calculations generated by the front end may therefore include multiplications by 4. The instruction set of the target machine may also favor certain layouts of data objects, and hence their addresses. We ignore alignment of data objects here; Example 7.3 shows how data objects are aligned by two compilers.

Declarations in a Procedure

The syntax of languages such as C, Pascal, and Fortran, allows all the declarations in a single procedure to be processed as a group. In this case, a global variable, say *offset*, can keep track of the next available relative address.

In the translation scheme of Fig. 8.11 nonterminal *P* generates a sequence of declarations of the form *id* : *T*. Before the first declaration is considered, *offset* is set to 0. As each new name is seen, that name is entered in the symbol table with offset equal to the current value of *offset*, and *offset* is incremented by the width of the data object denoted by that name.

The procedure *enter(name, type, offset)* creates a symbol-table entry for *name*, gives it type *type* and relative address *offset* in its data area. We use synthesized attributes *type* and *width* for nonterminal *T* to indicate the type and width, or number of memory units taken by objects of that type. Attribute *type* represents a type expression constructed from the basic types *integer* and *real* by applying the type constructors *pointer* and *array*, as in Section 6.1. If type expressions are represented by graphs, then attribute *type* might be a pointer to the node representing a type expression.

In Fig. 8.11, integers have width 4 and reals have width 8. The width of an array is obtained by multiplying the width of each element by the number of elements in the array.² The width of each pointer is assumed to be 4. In

² For arrays whose lower bound is not 0, the calculation of addresses for array elements is simplified if the offset entered into the symbol table is adjusted as discussed in Section 8.3.

$P \rightarrow$	$\{ \text{offset} := 0 \}$
D	
$D \rightarrow D ; D$	
$D \rightarrow \text{id} : T$	$\{ \text{enter}(\text{id.name}, T.\text{type}, \text{offset});$ $\text{offset} := \text{offset} + T.\text{width} \}$
$T \rightarrow \text{integer}$	$\{ T.\text{type} := \text{integer};$ $T.\text{width} := 4 \}$
$T \rightarrow \text{real}$	$\{ T.\text{type} := \text{real};$ $T.\text{width} := 8 \}$
$T \rightarrow \text{array} [\text{num}] \text{ of } T_1$	$\{ T.\text{type} := \text{array}(\text{num.val}, T_1.\text{type});$ $T.\text{width} := \text{num.val} \times T_1.\text{width} \}$
$T \rightarrow \uparrow T_1$	$\{ T.\text{type} := \text{pointer}(T_1.\text{type});$ $T.\text{width} := 4 \}$

Fig. 8.11. Computing the types and relative addresses of declared names.

Pascal and C, a pointer may be seen before we learn the type of the object pointed to (see the discussion of recursive types in Section 6.3). Storage allocation for such types is simpler if all pointers have the same width.

The initialization of *offset* in the translation scheme of Fig. 8.11 is more evident if the first production appears on one line as:

$$P \rightarrow \{ \text{offset} := 0 \} D \quad (8.2)$$

Nonterminals generating ϵ , called marker nonterminals in Section 5.6, can be used to rewrite productions so that all actions appear at the ends of right sides. Using a marker nonterminal *M*, (8.2) can be restated as:

$$\begin{aligned} P &\rightarrow M D \\ M &\rightarrow \epsilon \quad \{ \text{offset} := 0 \} \end{aligned}$$

Keeping Track of Scope Information

In a language with nested procedures, names local to each procedure can be assigned relative addresses using the approach of Fig. 8.11. When a nested procedure is seen, processing of declarations in the enclosing procedure is temporarily suspended. This approach will be illustrated by adding semantic rules to the following language.

$$\begin{aligned} P &\rightarrow D \\ D &\rightarrow D ; D \mid \text{id} : T \mid \text{proc id} ; D ; S \end{aligned} \quad (8.3)$$

The productions for nonterminals S for statements and T for types are not shown because we focus on declarations. The nonterminal T has synthesized attributes *type* and *width*, as in the translation scheme of Fig. 8.11.

For simplicity, suppose that there is a separate symbol table for each procedure in the language (8.3). One possible implementation of a symbol table is a linked list of entries for names. Clever implementations can be substituted if desired.

A new symbol table is created when a procedure declaration $D \rightarrow \text{proc id } D_1 ; S$ is seen, and entries for the declarations in D_1 are created in the new table. The new table points back to the symbol table of the enclosing procedure; the name represented by *id* itself is local to the enclosing procedure. The only change from the treatment of variable declarations in Fig. 8.11 is that the procedure *enter* is told which symbol table to make an entry in.

For example, symbol tables for five procedures are shown in Fig. 8.12. The nesting structure of the procedures can be deduced from the links between the symbol tables; the program is in Fig. 7.22. The symbol tables for procedures *readarray*, *exchange*, and *quicksort* point back to that for the containing procedure *sort*, consisting of the entire program. Since *partition* is declared within *quicksort*, its table points to that of *quicksort*.

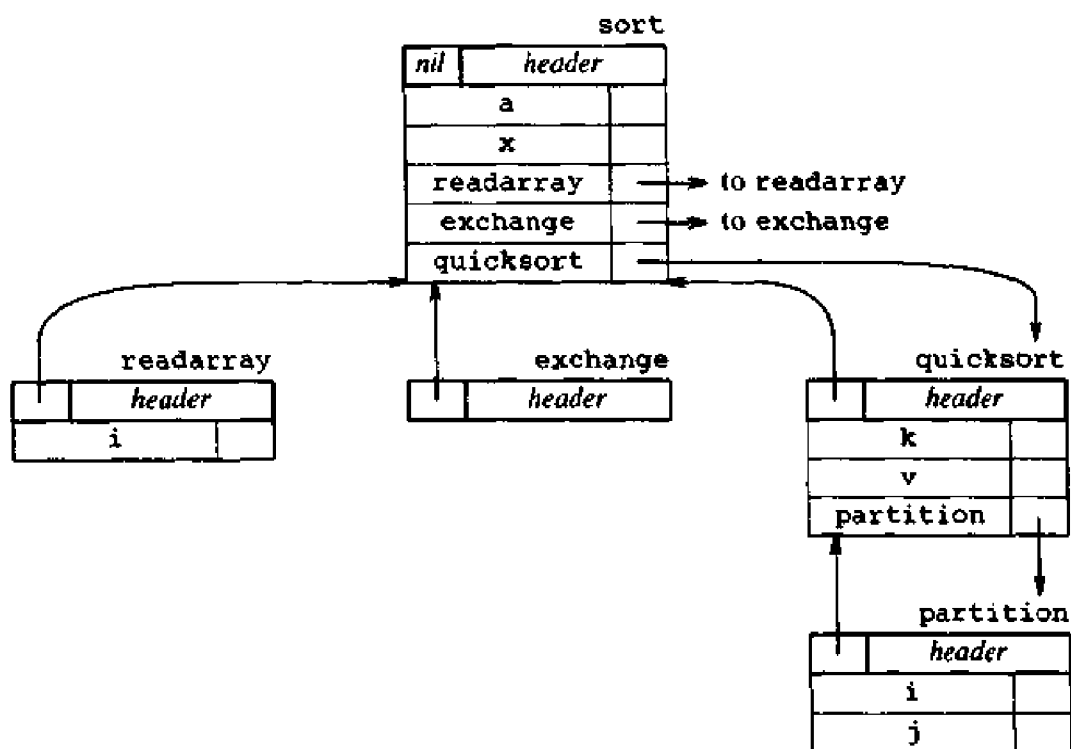


Fig. 8.12. Symbol tables for nested procedures.

The semantic rules are defined in terms of the following operations:

1. *mktable(previous)* creates a new symbol table and returns a pointer to the new table. The argument *previous* points to a previously created symbol table, presumably that for the enclosing procedure. The pointer *previous* is placed in a header for the new symbol table, along with additional information such as the nesting depth of a procedure. We can also number the procedures in the order they are declared and keep this number in the header.
2. *enter(table, name, type, offset)* creates a new entry for name *name* in the symbol table pointed to by *table*. Again, *enter* places type *type* and relative address *offset* in fields within the entry.
3. *addwidth(table, width)* records the cumulative width of all the entries in *table* in the header associated with this symbol table.
4. *enterproc(table, name, newtable)* creates a new entry for procedure *name* in the symbol table pointed to by *table*. The argument *newtable* points to the symbol table for this procedure *name*.

The translation scheme in Fig. 8.13 shows how data can be laid out in one pass, using a stack *tblptr* to hold pointers to symbol tables of the enclosing procedures. With the symbol tables of Fig. 8.12, *tblptr* will contain pointers to the tables for *sort*, *quicksort*, and *partition* when the declarations in *partition* are considered. The pointer to the current symbol table is on top. The other stack *offset* is the natural generalization to nested procedures of attribute *offset* in Fig. 8.11. The top element of *offset* is the next available relative address for a local of the current procedure.

All semantic actions in the subtrees for *B* and *C* in

$$A \rightarrow BC \{ action_A \}$$

are done before *action_A* at the end of the production occurs. Hence, the action associated with the marker *M* in Fig. 8.13 is the first to be done.

The action for nonterminal *M* initializes stack *tblptr* with a symbol table for the outermost scope, created by operation *mktable(nil)*. The action also pushes relative address 0 onto stack *offset*. The nonterminal *N* plays a similar role when a procedure declaration appears. Its action uses the operation *mktable(top(tblptr))* to create a new symbol table. Here, the argument *top(tblptr)* gives the enclosing scope of the new table. A pointer to the new table is pushed above that for the enclosing scope. Again, 0 is pushed onto *offset*.

For each variable declaration *id : T*, an entry is created for *id* in the current symbol table. This declaration leaves the stack *tblptr* unchanged; the top of stack *offset* is incremented by *T.width*. When the action on the right side of $D \rightarrow \text{proc } id; ND_1; S$ occurs, the width of all declarations generated by *D₁* is on top of stack *offset*; it is recorded using *addwidth*. Stacks *tblptr* and *offset*

$P \rightarrow M D$	$\{ \text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset}));$ $\text{pop}(\text{tblptr}); \text{pop}(\text{offset}) \}$
$M \rightarrow \epsilon$	$\{ t := \text{mktable}(\text{nil});$ $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$
$D \rightarrow D_1 ; D_2$	
$D \rightarrow \text{proc } \text{id} : N D_1 ; S$	$\{ t := \text{top}(\text{tblptr});$ $\text{addwidth}(t, \text{top}(\text{offset}));$ $\text{pop}(\text{tblptr}); \text{pop}(\text{offset});$ $\text{enterproc}(\text{top}(\text{tblptr}), \text{id.name}, t) \}$
$D \rightarrow \text{id} : T$	$\{ \text{enter}(\text{top}(\text{tblptr}), \text{id.name}, T.\text{type}, \text{top}(\text{offset}));$ $\text{top}(\text{offset}) := \text{top}(\text{offset}) + T.\text{width} \}$
$N \rightarrow \epsilon$	$\{ t := \text{mktable}(\text{top}(\text{tblptr}));$ $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$

Fig. 8.13. Processing declarations in nested procedures.

are then popped, and we revert to examining the declarations in the enclosing procedure. At this point, the name of the enclosed procedure is entered into the symbol table of its enclosing procedure.

Field Names in Records

The following production allows nonterminal T to generate records in addition to basic types, pointers, and arrays:

$T \rightarrow \text{record } D \text{ end}$

The actions in the translation scheme of Fig. 8.14 emphasize the similarity between the layout of records as a language construct and activation records. Since procedure definitions do not affect the width computations in Fig. 8.13, we overlook the fact that the above production also allows procedure definitions to appear within records.

$T \rightarrow \text{record } L D \text{ end}$	$\{ T.\text{type} := \text{record}(\text{top}(\text{tblptr}));$ $T.\text{width} := \text{top}(\text{offset});$ $\text{pop}(\text{tblptr}); \text{pop}(\text{offset}) \}$
$L \rightarrow \epsilon$	$\{ t := \text{mktable}(\text{nil});$ $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$

Fig. 8.14. Setting up a symbol table for field names in a record.

After the keyword **record** is seen, the action associated with the marker L

creates a new symbol table for the field names. A pointer to this symbol table is pushed onto stack *tblptr* and relative address 0 is pushed onto stack *offset*. The action for $D \rightarrow id : T$ in Fig. 8.13 therefore enters information about the field name *id* into the symbol table for the record. Furthermore, the top of stack *offset* will hold the width of all the data objects within the record after the fields have been examined. The action following **end** in Fig. 8.14 returns this width as synthesized attribute *T.width*. The type *T.type* is obtained by applying the constructor *record* to the pointer to the symbol table for this record. This pointer will be used in the next section to recover the names, types, and widths of the fields in the record from *T.type*.

8.3 ASSIGNMENT STATEMENTS

Expressions can be of type integer, real, array, and record in this section. As part of the translation of assignments into three-address code, we show how names can be looked up in the symbol table and how elements of arrays and records can be accessed.

Names in the Symbol Table

In Section 8.1, we formed three-address statements using names themselves, with the understanding that the names stood for pointers to their symbol-table entries. The translation scheme in Fig. 8.15 shows how such symbol-table entries can be found. The lexeme for the name represented by *id* is given by attribute *id.name*. Operation *lookup(id.name)* checks if there is an entry for this occurrence of the name in the symbol table. If so, a pointer to the entry is returned; otherwise, *lookup* returns *nil* to indicate that no entry was found.

The semantic actions in Fig. 8.15 use procedure *emit* to emit three-address statements to an output file, rather than building up *code* attributes for nonterminals, as in Fig. 8.6. From Section 2.3, translation can be done by emitting to an output file if the *code* attributes of the nonterminals on the left sides of productions are formed by concatenating the *code* attributes of the nonterminals on the right, in the same order that the nonterminals appear on the right side, perhaps with some additional strings in between.

By reinterpreting the *lookup* operation in Fig. 8.15, the translation scheme can be used even if the most closely nested scope rule applies to nonlocal names, as in Pascal. For concreteness, suppose that the context in which an assignment appears is given by the following grammar.

$$\begin{aligned}
 P &\rightarrow M D \\
 M &\rightarrow \epsilon \\
 D &\rightarrow D ; D \mid id : T \mid \text{proc } id ; N D ; S \\
 N &\rightarrow \epsilon
 \end{aligned}$$

Nonterminal *P* becomes the new start symbol when these productions are

$$\begin{aligned}
 S \rightarrow \text{id} := E & \quad \{ p := \text{lookup}(\text{id.name}); \\
 & \quad \text{if } p \neq \text{nil} \text{ then} \\
 & \quad \quad \text{emit}(p := E.\text{place}) \\
 & \quad \text{else error} \} \\
 E \rightarrow E_1 + E_2 & \quad \{ E.\text{place} := \text{newtemp}; \\
 & \quad \text{emit}(E.\text{place} := E_1.\text{place} + E_2.\text{place}) \} \\
 E \rightarrow E_1 * E_2 & \quad \{ E.\text{place} := \text{newtemp}; \\
 & \quad \text{emit}(E.\text{place} := E_1.\text{place} * E_2.\text{place}) \} \\
 E \rightarrow - E_1 & \quad \{ E.\text{place} := \text{newtemp}; \\
 & \quad \text{emit}(E.\text{place} := \text{'uminus'} E_1.\text{place}) \} \\
 E \rightarrow (E_1) & \quad \{ E.\text{place} := E_1.\text{place} \} \\
 E \rightarrow \text{id} & \quad \{ p := \text{lookup}(\text{id.name}); \\
 & \quad \text{if } p \neq \text{nil} \text{ then} \\
 & \quad \quad E.\text{place} := p \\
 & \quad \text{else error} \}
 \end{aligned}$$

Fig. 8.15. Translation scheme to produce three-address code for assignments.

added to those in Fig. 8.15.

For each procedure generated by this grammar, the translation scheme in Fig. 8.13 sets up a separate symbol table. Each such symbol table has a header containing a pointer to the table for the enclosing procedure. (See Fig. 8.12 for an example.) When the statement forming a procedure body is examined, a pointer to the symbol table for the procedure appears on top of the stack *tblptr*. This pointer is pushed onto the stack by actions associated with the marker nonterminal *N* on the right side of $D \rightarrow \text{proc id}; ND_1; S$.

Let the productions for nonterminal *S* be those in Fig. 8.15. Names in an assignment generated by *S* must have been declared in either the procedure that *S* appears in, or in some enclosing procedure. When applied to *name*, the modified *lookup* operation first checks if *name* appears in the current symbol table, accessible through *top(tblptr)*. If not, *lookup* uses the pointer in the header of a table to find the symbol table for the enclosing procedure and looks for the name there. If the name cannot be found in any of these scopes, then *lookup* returns *nil*.

For example, suppose that the symbol tables are as in Fig. 8.12 and that an assignment in the body of procedure *partition* is being examined. Operation *lookup(i)* will find an entry in the symbol table for *partition*. Since *v* is not in this symbol table, *lookup(v)* will use the pointer in the header in this symbol table to continue the search in the symbol table for the enclosing procedure *quicksort*.

Reusing Temporary Names

We have been going along assuming that *newtemp* generates a new temporary name each time a temporary is needed. It is useful, especially in optimizing compilers, to actually create a distinct name each time *newtemp* is called; Chapter 10 gives justification for doing so. However, the temporaries used to hold intermediate values in expression calculations tend to clutter up the symbol table, and space has to be allocated to hold their values.

Temporaries can be reused by changing *newtemp*. An alternative approach of packing distinct temporaries into the same location during code generation is explored in the next chapter.

The bulk of temporaries denoting data are generated during the syntax-directed translation of expressions, by rules such as those in Fig. 8.15. The code generated by the rules for $E \rightarrow E_1 + E_2$ has the general form:

```

evaluate  $E_1$  into  $t_1$ 
evaluate  $E_2$  into  $t_2$ 
 $t := t_1 + t_2$ 

```

From the rules for the synthesized attribute *E.place* it follows that t_1 and t_2 are not used elsewhere in the program. The lifetimes of these temporaries are nested like matching pairs of balanced parentheses. In fact, the lifetimes of all temporaries used in the evaluation of E_2 are contained in the lifetime of t_1 . It is therefore possible to modify *newtemp* so that it uses, as if it were a stack, a small array in a procedure's data area to hold temporaries.

Let us assume for simplicity that we are dealing only with integers. Keep a count *c*, initialized to zero. Whenever a temporary name is used as an operand, decrement *c* by 1. Whenever a new temporary name is generated, use $\$c$ and increase *c* by 1. Note that the "stack" of temporaries is not pushed or popped at run time, although it happens that stores and loads of temporary values are made by the compiler to occur at the "top."

Example 8.1. Consider the assignment

```
 $x := a * b + c * d - e * f$ 
```

Figure 8.16 shows the sequence of three-address statements that would be generated by semantic rules in Fig. 8.15, if *newtemp* were modified. The figure also contains an indication of the "current" value of *c* after the generation of each statement. Note that when we compute $\$0 - \1 , *c* is decremented to zero, so $\$0$ is again available to hold the result. □

Temporaries that may be assigned and/or used more than once, for example, in a conditional assignment, cannot be assigned names in the last-in first-out manner described above. Since they tend to be rare, all such temporary values can be assigned names of their own. The same problem of temporaries defined or used more than once occurs when we perform code optimization such as combining common subexpressions or moving a computation out of a loop (see Chapter 10). A reasonable strategy is to create a new name

STATEMENT	VALUE OF <i>c</i>
	0
\$0 := a + b	1
\$1 := c + d	2
\$0 := \$0 + \$1	1
\$1 := e + f	2
\$0 := \$0 - \$1	1
x := \$0	0

Fig. 8.16. Three-address code with stacked temporaries.

whenever we create an additional definition or use for a temporary or move its computation.

Addressing Array Elements

Elements of an array can be accessed quickly if the elements are stored in a block of consecutive locations. If the width of each array element is w , then the i th element of array A begins in location

$$base + (i - low) \times w \quad (8.4)$$

where low is the lower bound on the subscript and $base$ is the relative address of the storage allocated for the array. That is, $base$ is the relative address of $A[low]$.

The expression (8.4) can be partially evaluated at compile time if it is rewritten as

$$i \times w + (base - low \times w)$$

The subexpression $c = base - low \times w$ can be evaluated when the declaration of the array is seen. We assume that c is saved in the symbol table entry for A , so the relative address of $A[i]$ is obtained by simply adding $i \times w$ to c .

Compile-time precalculation can also be applied to address calculations of elements of multi-dimensional arrays. A two-dimensional array is normally stored in one of two forms, either *row-major* (row-by-row) or *column-major* (column-by-column). Figure 8.17 shows the layout of a 2×3 array A in (a) row-major form and (b) column-major form. Fortran uses column-major form; Pascal uses row-major form, because $A[i, j]$ is equivalent to $A[i][j]$, and the elements of each array $A[i]$ are stored consecutively.

In the case of a two-dimensional array stored in row-major form, the relative address of $A[i_1, i_2]$ can be calculated by the formula

$$base + ((i_1 - low_1) \times n_2 + i_2 - low_2) \times w$$

where low_1 and low_2 are the lower bounds on the values of i_1 and i_2 and n_2 is the number of values that i_2 can take. That is, if $high_2$ is the upper bound

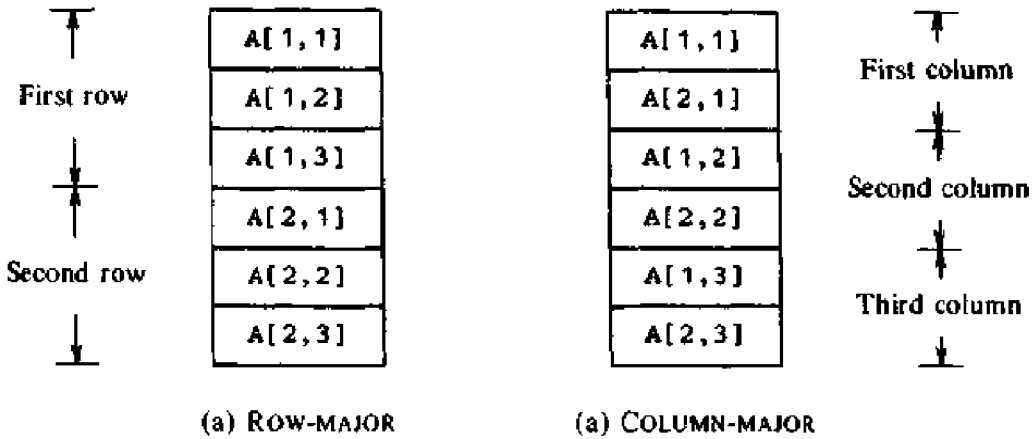


Fig. 8.17. Layouts for a two-dimensional array.

on the value of i_2 , then $n_2 = \text{high}_2 - \text{low}_2 + 1$. Assuming that i_1 and i_2 are the only values that are not known at compile time, we can rewrite the above expression as

$$((i_1 \times n_2) + i_2) \times w + (\text{base} - ((\text{low}_1 \times n_2) + \text{low}_2) \times w) \quad (8.5)$$

The last term in this expression can be determined at compile time.

We can generalize row- or column-major form to many dimensions. The generalization of row-major form is to store the elements in such a way that, as we scan down a block of storage, the rightmost subscripts appear to vary fastest, like the numbers on an odometer. The expression (8.5) generalizes to the following expression for the relative address of $A[i_1, i_2, \dots, i_k]$

$$((\dots ((i_1 n_2 + i_2) n_3 + i_3) \dots) n_k + i_k) \times w + \text{base} - ((\dots ((\text{low}_1 n_2 + \text{low}_2) n_3 + \text{low}_3) \dots) n_k + \text{low}_k) \times w \quad (8.6)$$

Since for all j , $n_j = \text{high}_j - \text{low}_j + 1$ is assumed fixed, the term on the second line of (8.6) can be computed by the compiler and saved with the symbol-table entry for A .³ Column-major form generalizes to the opposite arrangement, with the leftmost subscripts varying fastest.

Some languages permit the sizes of arrays to be specified dynamically, when a procedure is called at run-time. The allocation of such arrays on a run-time stack was considered in Section 7.3. The formulas for accessing the elements of such arrays are the same as for fixed-size arrays, but the upper and lower limits are not known at compile time.

³ In C, a multi-dimensional array is simulated by defining arrays whose elements are arrays. For example, suppose x is an array of arrays of integers. Then, the language allows both $x[i]$ and $x[i][j]$ to be written, and the widths of these expressions are different. However, the lower bound of all arrays is 0, so the term on the second line of (8.6) simplifies to base in each case.

The chief problem in generating code for array references is to relate the computation of (8.6) to a grammar for array references. Array references can be permitted in assignments if nonterminal L with the following productions is allowed where id appears in Fig. 8.15:

$$\begin{aligned} L &\rightarrow \text{id} [\text{Elist}] \mid \text{id} \\ \text{Elist} &\rightarrow \text{Elist} , E \mid E \end{aligned}$$

In order that the various dimensional limits n_j of the array be available as we group index expressions into an *Elist*, it is useful to rewrite the productions as

$$\begin{aligned} L &\rightarrow \text{Elist}] \mid \text{id} \\ \text{Elist} &\rightarrow \text{Elist} , E \mid \text{id} [E \end{aligned}$$

That is, the array name is attached to the leftmost index expression rather than being joined to *Elist* when an L is formed. These productions allow a pointer to the symbol-table entry for the array name to be passed as a synthesized attribute *array* of *Elist*.⁴

We also use *Elist.ndim* to record the number of dimensions (index expressions) in the *Elist*. The function *limit(array, j)* returns n_j , the number of elements along the j th dimension of the array whose symbol-table entry is pointed to by *array*. Finally, *Elist.place* denotes the temporary holding a value computed from index expressions in *Elist*.

An *Elist* that produces the first m indices of a k -dimensional array reference $A[i_1, i_2, \dots, i_k]$ will generate three-address code to compute

$$(\dots ((i_1 n_2 + i_2) n_3 + i_3) \dots) n_m + i_m \quad (8.7)$$

using the recurrence

$$\begin{aligned} e_1 &= i_1 \\ e_m &= e_{m-1} \times n_m + i_m \end{aligned} \quad (8.8)$$

Thus, when $m = k$, a multiplication by the width w is all that will be needed to compute the term on the first line of (8.6). Note that the i_j 's here may really be values of expressions, and code to evaluate those expressions will be interspersed with code to compute (8.7).

An l -value L will have two attributes, *L.place* and *L.offset*. In the case that L is a simple name, *L.place* will be a pointer to the symbol-table entry for that name, and *L.offset* will be **null**, indicating that the l -value is a simple name rather than an array reference. The nonterminal E has the same translation *E.place*, with the same meaning as in Fig 8.15.

⁴ The transformation is similar to one mentioned at the end of Section 5.6 for eliminating inherited attributes. Here too, we could have solved the problem with inherited attributes.

The Translation Scheme for Addressing Array Elements

Semantic actions will be added to the grammar:

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow Elist \]$
- (6) $L \rightarrow id$
- (7) $Elist \rightarrow Elist \ , \ E$
- (8) $Elist \rightarrow id \ [\ E$

As in the case of expressions without array references, the three-address code itself is produced by the *emit* procedure invoked in the semantic actions.

We generate a normal assignment if L is a simple name, and an indexed assignment into the location denoted by L otherwise:

- (1) $S \rightarrow L := E$ { if $L.offset = \text{null}$ then /* L is a simple id */
 $emit(L.place \ ' := \ E.place);$
 else
 $emit(L.place \ [\ 'L.offset' \] \ ' := \ E.place) \ }$

The code for arithmetic expressions is exactly the same as in Fig. 8.15:

- (2) $E \rightarrow E_1 + E_2$ { $E.place := \text{newtemp};$
 $emit(E.place \ ' := \ E_1.place \ '+' \ E_2.place) \ }$
- (3) $E \rightarrow (E_1)$ { $E.place := E_1.place \ }$

When an array reference L is reduced to E , we want the r -value of L . Therefore we use indexing to obtain the contents of the location $L.place[L.offset]$:

- (4) $E \rightarrow L$ { if $L.offset = \text{null}$ then /* L is a simple id */
 $E.place := L.place$
 else begin
 $E.place := \text{newtemp};$
 $emit(E.place \ ' := \ L.place \ [\ 'L.offset' \] \)$
 end }

Below, $L.offset$ is a new temporary representing the first term of (8.6); function $width(Elist.array)$ returns w in (8.6). $L.place$ represents the second term of (8.6), returned by the function $c(Elist.array)$.

- (5) $L \rightarrow Elist \]$ { $L.place := \text{newtemp};$
 $L.offset := \text{newtemp};$
 $emit(L.place \ ' := \ c(Elist.array));$
 $emit(L.offset \ ' := \ Elist.place \ '*' \ width(Elist.array)) \ }$

A null offset indicates a simple name.

(6) $L \rightarrow \text{id}$ $\{ L.place := \text{id.place};$
 $L.offset := \text{null} \}$

When the next index expression is seen, we apply the recurrence (8.8). In the following action, $Elist_1.place$ corresponds to e_{m-1} in (8.8) and $Elist.place$ to e_m . Note that if $Elist_1$ has $m-1$ components, then $Elist$ on the left side of the production has m components.

(7) $Elist \rightarrow Elist_1, E$ $\{ t := \text{newtemp};$
 $m := Elist_1.ndim + 1;$
 $\text{emit}(t' := 'Elist_1.place' * \text{limit}(Elist_1.array, m));$
 $\text{emit}(t' := t' + 'E.place');$
 $Elist.array := Elist_1.array;$
 $Elist.place := t;$
 $Elist.ndim := m \}$

$E.place$ holds both the value of the expression E and the value of (8.7) for $m=1$.

(8) $Elist \rightarrow \text{id} [E$ $\{ Elist.array := \text{id.place};$
 $Elist.place := E.place;$
 $Elist.ndim := 1 \}$

Example 8.2. Let A be a 10×20 array with $low_1 = low_2 = 1$. Therefore, $n_1 = 10$ and $n_2 = 20$. Take w to be 4. An annotated parse tree for the assignment $x := A[y, z]$ is shown in Fig. 8.18. The assignment is translated into the following sequence of three-address statements:

```

t1 := y * 20
t1 := t1 + z
t2 := c           /* constant c = baseA - 84 */
t3 := 4 * t1
t4 := t2[t3]
x := t4

```

For each variable, we have used its name in place of id.place . □

Type Conversions within Assignments

In practice, there would be many different types of variables and constants, so the compiler must either reject certain mixed-type operations or generate appropriate coercion (type conversion) instructions.

Consider the grammar for assignment statements as above, but suppose there are two types — real and integer, with integers converted to reals when


```

E.place := newtemp;
if E1.type = integer and E2.type = integer then begin
    emit(E.place ' := ' E1.place 'int+' E2.place);
    E.type := integer
end
else if E1.type = real and E2.type = real then begin
    emit(E.place ' := ' E1.place 'real+' E2.place);
    E.type := real
end
else if E1.type = integer and E2.type = real then begin
    u := newtemp;
    emit(u ' := ' 'inttoreal' E1.place);
    emit(E.place ' := ' u 'real+' E2.place);
    E.type := real
end
else if E1.type = real and E2.type = integer then begin
    u := newtemp;
    emit(u ' := ' 'inttoreal' E2.place);
    emit(E.place ' := ' E1.place 'real+' u);
    E.type := real
end
else
    E.type := type_error;

```

Fig. 8.19. Semantic action for $E \rightarrow E_1 + E_2$.

For example, for the input

```
x := y + i * j
```

assuming *x* and *y* have type *real*, and *i* and *j* have type *integer*, the output would look like

```

t1 := i int* j
t3 := inttoreal t1
t2 := y real+ t3
x := t2

```

The semantic action of Fig. 8.19 uses two attributes *E.place* and *E.type* for the nonterminal *E*. As the number of types subject to conversion increases, the number of cases that arise increases quadratically (or worse, if there are operators with more than two arguments). Therefore with large numbers of types, careful organization of the semantic actions becomes more important.

Accessing Fields in Records

The compiler must keep track of both the types and relative addresses of the fields of a record. An advantage of keeping this information in symbol-table entries for the field names is that the routine for looking up names in the symbol table can also be used for field names. With this in mind, a separate symbol table was created for each record type by the semantic actions in Fig. 8.14 in the last section. If t is a pointer to the symbol table for a record type, then the type $record(t)$ formed by applying the constructor $record$ to the pointer was returned as $T.type$.

We use the expression

$$p↑.info + 1$$

to illustrate how a pointer to the symbol table can be extracted from an attribute $E.type$. From the operations in this expression it follows that p must be a pointer to a record with a field name `info` whose type is arithmetic. If types are constructed as in Fig. 8.13 and 8.14, the type of p must be given by a type expression

$$pointer(record(t))$$

The type of $p↑$ is then $record(t)$, from which t can be extracted. The field name `info` is looked up in the symbol table pointed to by t .

8.4 BOOLEAN EXPRESSIONS

In programming languages, boolean expressions have two primary purposes. They are used to compute logical values, but more often they are used as conditional expressions in statements that alter the flow of control, such as *if-then*, *if-then-else*, or *while-do* statements.

Boolean expressions are composed of the boolean operators (**and**, **or**, and **not**) applied to elements that are boolean variables or relational expressions. In turn, relational expressions are of the form $E_1 \text{ relop } E_2$, where E_1 and E_2 are arithmetic expressions. Some languages, such as PL/I, allow more general expressions, where boolean, arithmetic, and relational operators can be applied to expressions of any type whatever, with no distinction between boolean and arithmetic values; a coercion is performed when necessary. In this section, we consider boolean expressions generated by the following grammar:

$$E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \text{not } E \mid (E) \mid \text{id relop id} \mid \text{true} \mid \text{false}$$

We use the attribute op to determine which of the comparison operators $<$, $\leq$, $=$, $\neq$, $>$, or $\geq$ is represented by **relop**. As is customary, we assume that **or** and **and** are left-associative, and that **or** has lowest precedence, then **and**, then **not**.

Methods of Translating Boolean Expressions

There are two principal methods of representing the value of a boolean expression. The first method is to encode true and false numerically and to evaluate a boolean expression analogously to an arithmetic expression. Often 1 is used to denote true and 0 to denote false, although many other encodings are also possible. For example, we could let any nonzero quantity denote true and zero denote false, or we could let any nonnegative quantity denote true and any negative number denote false.

The second principal method of implementing boolean expressions is by flow of control, that is, representing the value of a boolean expression by a position reached in a program. This method is particularly convenient in implementing the boolean expressions in flow-of-control statements, such as the if-then and while-do statements. For example, given the expression $E_1 \text{ or } E_2$, if we determine that E_1 is true, then we can conclude that the entire expression is true without having to evaluate E_2 .

The semantics of the programming language determines whether all parts of a boolean expression must be evaluated. If the language definition permits (or requires) portions of a boolean expression to go unevaluated, then the compiler can optimize the evaluation of boolean expressions by computing only enough of an expression to determine its value. Thus, in an expression such as $E_1 \text{ or } E_2$, neither E_1 nor E_2 is necessarily evaluated fully. If either E_1 or E_2 is an expression with side effects (e.g., contains a function that changes a global variable), then an unexpected answer may be obtained.

Neither of the above methods is uniformly superior to the other. For example, the BLISS/11 optimizing compiler (Wulf et al. 1975), among others, chooses the appropriate method for each expression individually. This section considers both methods for the translation of boolean expressions to three-address code.

Numerical Representation

Let us first consider the implementation of boolean expressions using 1 to denote true and 0 to denote false. Expressions will be evaluated completely, from left to right, in a manner similar to arithmetic expressions. For example, the translation for

$a \text{ or } b \text{ and not } c$

is the three-address sequence

```
t1 := not c
t2 := b and t1
t3 := a or t2
```

A relational expression such as $a < b$ is equivalent to the conditional statement **if** $a < b$ **then** 1 **else** 0, which can be translated into the three-address code sequence (again, we arbitrarily start statement numbers at 100):

```

100:  if a < b goto 103
101:  t := 0
102:  goto 104
103:  t := 1
104:

```

A translation scheme for producing three-address code for boolean expressions is shown in Fig. 8.20. In this scheme, we assume that *emit* places three-address statements into an output file in the right format, that *nextstat* gives the index of the next three-address statement in the output sequence, and that *emit* increments *nextstat* after producing each three-address statement.

$E \rightarrow E_1 \text{ or } E_2$	{ $E.place := newtemp;$ $emit(E.place := E_1.place \text{ 'or' } E_2.place)$ }
$E \rightarrow E_1 \text{ and } E_2$	{ $E.place := newtemp;$ $emit(E.place := E_1.place \text{ 'and' } E_2.place)$ }
$E \rightarrow \text{not } E_1$	{ $E.place := newtemp;$ $emit(E.place := \text{'not' } E_1.place)$ }
$E \rightarrow (E_1)$	{ $E.place := E_1.place$ }
$E \rightarrow id_1 \text{ relop } id_2$	{ $E.place := newtemp;$ $emit(\text{'if' } id_1.place \text{ relop } id_2.place \text{ 'goto' nextstat} + 3);$ $emit(E.place := \text{'0'})$; $emit(\text{'goto' nextstat} + 2);$ $emit(E.place := \text{'1'})$ }
$E \rightarrow \text{true}$	{ $E.place := newtemp;$ $emit(E.place := \text{'1'})$ }
$E \rightarrow \text{false}$	{ $E.place := newtemp;$ $emit(E.place := \text{'0'})$ }

Fig. 8.20. Translation scheme using a numerical representation for booleans.

Example 8.3. The scheme in Fig. 8.20 would generate the three-address code in Fig. 8.21 for the expression $a < b \text{ or } c < d \text{ and } e < f$. $\square$

Short-Circuit Code

We can also translate a boolean expression into three-address code without generating code for any of the boolean operators and without having the code necessarily evaluate the entire expression. This style of evaluation is sometimes called “short-circuit” or “jumping” code. It is possible to evaluate boolean expressions without generating code for the boolean operators **and**, **or**, and **not** if we represent the value of an expression by a position in the code sequence. For example, in Fig. 8.21, we can tell what value t_1 will

100: if $a < b$ goto 103	107: $t_2 := 1$
101: $t_1 := 0$	108: if $e < f$ goto 111
102: goto 104	109: $t_3 := 0$
103: $t_1 := 1$	110: goto 112
104: if $c < d$ goto 107	111: $t_3 := 1$
105: $t_2 := 0$	112: $t_4 := t_2$ and t_3
106: goto 108	113: $t_5 := t_1$ or t_4

Fig. 8.21. Translation of $a < b$ or $c < d$ and $e < f$.

have by whether we reach statement 101 or statement 103, so the value of t_1 is redundant. For many boolean expressions, it is possible to determine the value of the expression without having to evaluate it completely.

Flow-of-Control Statements

We now consider the translation of boolean expressions into three-address code in the context of if-then, if-then-else, and while-do statements such as those generated by the following grammar:

$$\begin{array}{l}
 S \rightarrow \text{if } E \text{ then } S_1 \\
 \quad | \text{ if } E \text{ then } S_1 \text{ else } S_2 \\
 \quad | \text{ while } E \text{ do } S_1
 \end{array}$$

In each of these productions, E is the boolean expression to be translated. In the translation, we assume that a three-address statement can be symbolically labeled, and that the function *newlabel* returns a new symbolic label each time it is called.

With a boolean expression E , we associate two labels: $E.true$, the label to which control flows if E is true, and $E.false$, the label to which control flows if E is false. The semantic rules for translating a flow-of-control statement S allow control to flow from the translation $S.code$ to the three-address instruction immediately following $S.code$. In some cases, the instruction immediately following $S.code$ is a jump to some label L . A jump to a jump to L from within $S.code$ is avoided using an inherited attribute $S.next$. The value of $S.next$ is a label that is attached to the first three-address instruction to be executed after the code for S .⁵ The initialization of $S.next$ is not shown.

In translating the if-then statement $S \rightarrow \text{if } E \text{ then } S_1$, a new label $E.true$ is created and attached to the first three-address instruction generated for the statement S_1 , as in Fig. 8.22(a). A syntax-directed definition appears in Fig. 8.23. The code for E generates a jump to $E.true$ if E is true and a jump to $S.next$ if E is false. We therefore set $E.false$ to $S.next$.

⁵ If implemented literally, the approach of inheriting a label $S.next$ can lead to a proliferation of labels. The backpatching approach of Section 8.6 creates labels only when they are needed.

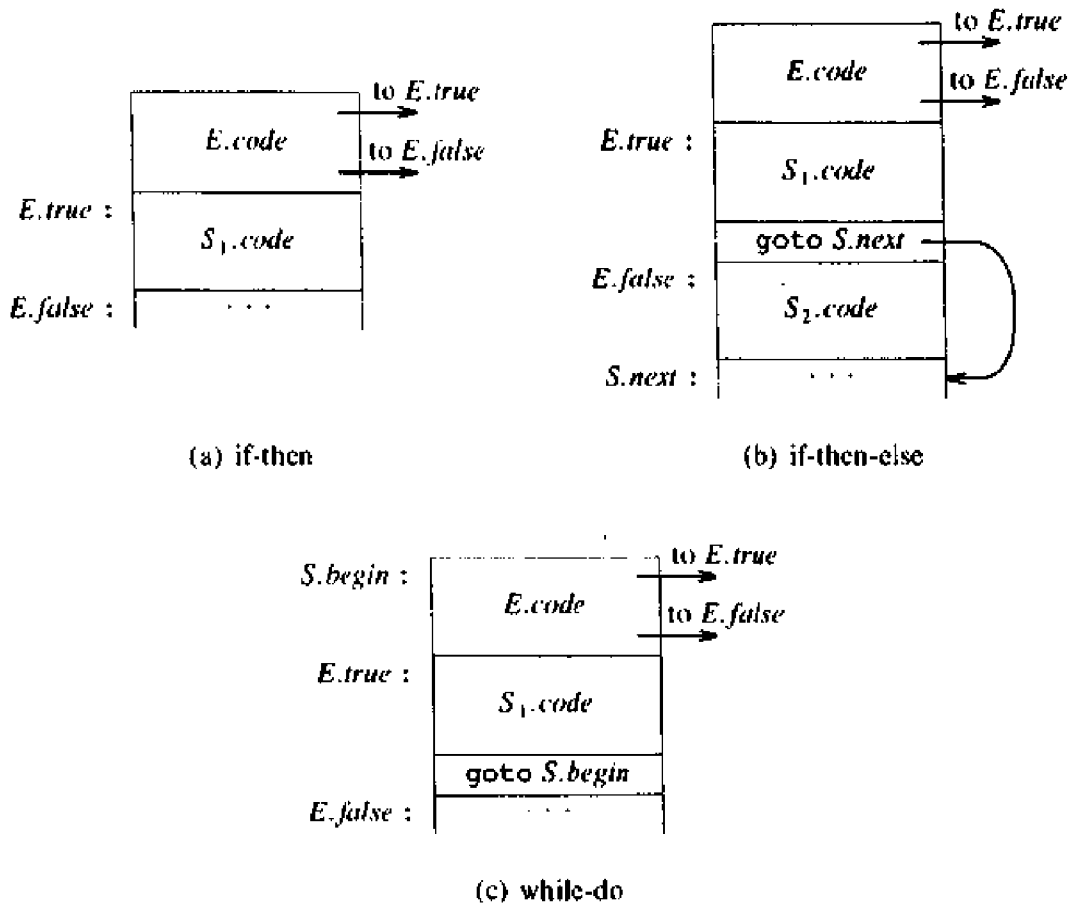


Fig. 8.22. Code for if-then, if-then-else, and while-do statements.

In translating the if-then-else statement $S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$, the code for the boolean expression E has jumps out of it to the first instruction of the code for S_1 if E is true, and to the first instruction of the code for S_2 if E is false, as illustrated in Fig. 8.22(b). As with the if-then statement, an inherited attribute $S.next$ gives the label of the three-address instruction to be executed next after executing the code for S . An explicit `goto S.next` appears after the code for S_1 , but not after S_2 . We leave it to the reader to show that, with these semantic rules, if $S.next$ is not the label of the instruction immediately following $S_2.code$, then an enclosing statement will supply the jump to label $S.next$ after the code for S_2 .

The code for $S \rightarrow \text{while } E \text{ do } S_1$ is formed as shown in Fig. 8.22(c). A new label $S.begin$ is created and attached to the first instruction generated for E . Another new label $E.true$ is attached to the first instruction for S_1 . The code for E generates a jump to this label if E is true, a jump to $S.next$ if E is false; again, we set $E.false$ to be $S.next$. After the code for S_1 we place the instruction `goto S.begin`, which causes a jump back to the beginning of the code for the boolean expression. Note that $S_1.next$ is set to this label $S.begin$, so jumps from within $S_1.code$ can go directly to $S.begin$.